6 · *Environmental Predictors: Issues of Processing and Selection*

Predictive habitat suitability modeling usually requires extensive access to resources from a GIS, and one of the difficulties in producing effective modeling lies in the need to combine statistical and GIS modeling. This difficulty has already been addressed, mostly in reference to the limited availability of statistical functions in GIS environments (Guisan and Zimmermann, 2000). While for large spatial data structures and extensive databases this limitation still exists to some extent, there have been major improvements over the last decade, primarily in terms of the functionality of standalone packages for habitat suitability modeling (e.g. Maxent, OpenModeller, GRASP) and the ever-developing statistical environment R. Many R packages can now deal with different types of spatial data, and although most GIS functionalities have been included in several packages, they often use different formats to store spatial objects. However, the field of spatial data handling and analysis in R is rapidly evolving, and this will make GIS access obsolete for many analytical paths. Here, we introduce some of these new functionalities primarily using the `raster` package. Although under continuous development, it offers many of the well-known raster functions usually used in ArcGIS or other GIS software, and it is therefore very well suited to numerous aspects of exploratory analyses and spatial modeling in R. Other packages used here are `maptools`, `sp`, `rasterVis`, and `rgdal` from CRAN.

6.1 Existing Environmental Databases

There are a large number of available datasets ranging from the global[1] to the local scale, and we will not attempt to provide an exhaustive list here.

[1] http://worldgrids.org/doku.php?id=source_data and http://freegisdata.rtwilson.com/index.html

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

Most readers will have their own databases anyway. However, there are a number of datasets that are very commonly used, so we feel it is helpful to list some of these data sources, as they are likely to be useful when used in combination with one's own data. These have been organized according to the nature of the data. Please note that these links change very frequently, and might already be out of date when this book goes to print.

6.1.1 Digital Elevation Data

Digital elevation data (DEM), a three-dimensional representation of a terrain's surface, is very useful for deriving altitude, slope, and aspect, for example. There are two datasets used extensively at global scale. The first is the relatively dated GLOBE[2] dataset (Hastings et al., 1999), also known as GTOPO30, which is available in geographic projection at a resolution of 30 arc seconds (~1 km resolution at the equator). This dataset provides a full global coverage, but is not very precise. Nonetheless, it is perfectly sufficient for most global modeling approaches. A similar dataset including bathymetry data is available as ETOPO1[3] (Amante and Eakins, 2009) at a resolution of 1 arc minute (roughly 2 km at the equator) in geographic projection.

For finer scale analyses, most researcher use their own data, yet the globally available 90m (30m for the US) dataset SRTM[4] is a helpful data source for many, especially since it offers seamless availability for all areas between 60°N and 60°S. This dataset is also available in geographic projection as 1° (lat/lon) tiles. It has been updated and improved compared to version four, mostly with regards to gap filling.

Another interesting and comparably recent source is the ASTER-DEM, available from the Jet Propulsion Lab.[5] It was derived from multi-angular ASTER space-borne imaging. It is available seamlessly for areas between 83°N and 83°S, and thus overcomes the data gap for northern latitude in the SRTM dataset. The spatial resolution is 30 m, and the data is made available in 1° tiles globally. Although the data is available at even finer resolution than the SRTM, it is not necessarily more precise. So we advise checking the accuracy carefully when it is applied to small spatial extents, and to resample the grids at a coarser resolution, if necessary.

[2] www.ngdc.noaa.gov/mgg/topo/gltiles.html
[3] www.ngdc.noaa.gov/mgg/global/global.html
[4] http://srtm.csi.cgiar.org/
[5] http://asterweb.jpl.nasa.gov/gdem.asp

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

There are many other national datasets, and most of these have restricted access or require registration. Consistent DEM datasets are very important in order to generate predictors for scaling, e.g. scaling coarser resolution climate data to finer resolutions. Most DEM data are available in ESRI grid format. Other useful formats are GeoTIFF or NetCDF. Global data are usually available in the WGS84 geographic coordinate reference system, while regional applications often have the spatial data transformed to projections that correct for angles or area. The latter is important for many ecological applications.

6.1.2 Climate Data

There are several large-scale datasets available, and depending on the nature of the analysis, researchers can choose the one dataset that is appropriate for their analysis. Worldclim[6] is probably the most widely used global climate dataset for ecological analyses (Hijmans et al., 2005). It is available in geographic projection at a spatial resolution of 30 arc seconds (~1 km), but also at coarser resolutions (2.5, 5, and 10 arc minutes). In order to project the climate extrapolation spatially, the SRTM dataset (see above) was used and resampled to the resolution and spatial registration of the GTOPO30 DEM where available, while the latter was used elsewhere. Worldclim maps are based on a large number of climate stations using long-term (1950–2000 in general, but 1961–1990 for most stations) monthly mean climate information for precipitation (47,554 stations), maximum (24,542 stations), and minimum (14,930 stations) temperature. It is generally available as version 1.4. Just recently, version 2.0 was released for beta testing (beta release 1, June 2016). Due to the globally uneven distribution of climate stations, the mapping uncertainty varies substantially in space. The climate mapping method is based on the ANUSPLIN package, which uses thin-plate smoothing splines (Hutchinson, 1995). In addition to basic climate parameters such as monthly mean, minimum, and maximum temperature and precipitation, this dataset provides a set of 19 so-called bioclimatic variables (bioclim), which are supposed to be more biological relevant than the original monthly climate layers, from which the bioclim variables are derived. The datasets are primarily made available for the current climate. However, the website also offers datasets for historical data (three time slices for the last interglacial, last glacial maximum, and mid-Holocene), as well as for projected future climates for large numbers of global circulation models (GCM) and scenarios

[6] http://worldclim.org/

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

originating from CMIP5 (Coupled Model Intercomparison Project 5; Meehl et al., 2009) and the Fifth Assessment Report (IPCC, 2013) of the Intergovernmental Panel on Climate Change (IPCC). Based on the Worldclim baseline (current) climate, the CGIAR (Consultative Group on International Agricultural Research) website operated by the Climate Change, Agriculture and Food Security research program (CCAFS) provides a large array of additional projected future climate datasets for 10-year monthly means.[7] These are available at different resolutions and are based on various downscaling techniques. They currently represent the largest available archive of downscaled climate data for use in ecology, evolution and environmental sciences.

For North America, there are three widely used datasets at similar spatial resolution. The first is the PRISM dataset.[8] This mapping approach is semi-statistical in that certain theoretically derived maximum lapse rates are not exceeded when calculating regressions from station values (Daly et al., 1994). Climate data is available for individual months and years at a 4 km spatial resolution from 1895–2010, while 1981–2010 climatological normals are available at an 800 m and a 4 km spatial resolution. PRISM also provides daily data (at 4 km resolution) since 1981. The DAYMET dataset[9] is based on daily climate interpolations (Thornton et al., 1997; Thornton and Running, 1999) originally available for the 48 conterminous United States and maps daily climate surfaces for minimum and maximum temperature, precipitation, global incoming shortwave radiation, and vapor pressure deficit for the period 1980–1997. The new version v3 now covers all of North America, Puerto Rico, and Hawaii and is available at a 1 km spatial resolution in the NetCDF format and covers the period of 1980–2015 (Thornton et al., 2016). Maps are also made available for monthly and yearly data. Point locations can be sampled on the DAYMET website for any sequence of days. The DAYMET method is based on a distance-weighted regression, where weights follow a Gaussian shape filter, which is adjusted on a yearly basis according to the availability of stations and the spatial density of these stations is also considered. Another set of climate layers is available from the Moscow Forestry Sciences Lab.[10] It is available at 30 arc second resolution and is based on the ANUSPLIN method (as Worldclim), but more emphasis was placed

[7] www.ccafs-climate.org/
[8] www.prism.oregonstate.edu/
[9] https://daymet.ornl.gov/
[10] http://forest.moscowfsl.wsu.edu/climate/

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

on regional climate station density. Products are available for the western United States, for Mexico, and for all of North America, including Mexico, the United States (including Alaska), and Canada. As in Worldclim, a set of 15 derived variables has been computed that is supposed to have more biological relevance. In addition to current climate layers, data are provided for 2–3 SRES scenarios from three GCMs for three time slices of an average of 10 years each representing simulations for the 4th IPCC Assessment Report (IPCC, 2007). Now, the site also offers 2–3 RCP scenarios for 5 GCMs (again three time slices of 10 years each) representing simulations for the 5th IPCC Assessment Report (IPCC, 2013). In addition, an ensemble of 17 GCMs for 3 RCPs is provided in the same temporal resolution as the datasets mentioned above.

While all the products described above are available at approximately 1 km spatial resolution, there are many more available at much coarser resolutions. At the 10' to 0.5° resolution, the most widely used dataset is the one produced by the climate research unit (CRU) of the University of East Anglia.[11] These datasets have coarser spatial resolutions, but provide time-series for both historical (1900–2000) and projected future (2001–2100) climates, where the latter are available for a range of IPCC scenarios and GCMs. The biggest source of projected future climate data is available from the CMIP data portal,[12] which currently hosts the most recent global GCM simulations (CMIP5) that were used for the 5th IPCC Assessment Report (IPCC, 2013). Based on these GCMs, a new effort to physically downscale these GCMs using regional climate models (RCMs) was initiated, known as CORDEX (Coordinated Regional Climate Downscaling Experiment), which offers global coverage for climate data at a 0.44 ° (50 km) spatial resolution.[13]

Such data are frequently used in combination with higher resolution climate data such as Worldclim in order to provide downscaled projections of future climate at a regional scale (e.g. Engler et al., 2009; Randin et al., 2009). While the CRU website provides a consistent set of data (variables, IPCC scenarios, GCMs, spatial resolution), the different versions of global GCM runs in the CMIP database are less consistent with regards to extent, spatial resolution, or temporal coverage. The CORDEX database is more consistent, as all projections are generated at the same spatial resolution. These datasets are usually stored in the

[11] www.cru.uea.ac.uk/cru/data/hrg/
[12] http://cmip-pcmdi.llnl.gov/index.html
[13] www.cordex.org/

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

NetCDF format, which is optimized for storing the complex data structures produced by GCMs or RCMs.

6.1.3 Land Cover/Land-Use Data

There are a large number of global land and vegetation cover datasets. While the different datasets were developed independently from each other and for a range of different purposes, more modern products are often provided with several classification schemes, and therefore include more or less classes for the same basic product. Some of these products are regularly updated, while others represent the land cover classification for input data from a certain time period. In this case, this data can only be used meaningfully if explicit reference is made to the time period in question.

Land-use is often reclassified from land cover datasets, and there are fewer products available. However, both types of datasets are important and useful for a range of large to regional scale applications. Both land-cover and land-use change are primary threats to regional biodiversity (Sala et al., 2000; Thuiller, 2007), and it is vital that information on these patterns and processes is included when assessing and modeling species patterns in space and time (Thuiller et al., 2014a). We will discuss herein a number of datasets of interest at regional or global scale, but this list is by no means exhaustive.

One of the early datasets was the International Geosphere Biosphere Program (IGBP) classification of land cover containing 17 classes (Belward et al., 1999). It represents a classification of data collected on a daily basis between April 1992 and March 1993 by the Advance Very High Resolution Radiometer (AVHRR) scanner, a satellite operated by the National Oceanic and Space Administration (NOAA). The first version of this dataset was released in 1997 as version 1.2, and an updated classification of the same original data was later made available as version 2.0.[14] This basic 17-item legend is now continued with a product from the MODIS sensor on board the TERRA satellite, available yearly since 2000 at 500 m and 0.05° spatial resolution.[15] A similar land cover product was generated using AVHRR data and a classification scheme of 12 classes globally (Hansen et al., 2000),[16] available at 1 and 8 km, as well as at 1° spatial resolution. MODIS data can be obtained from an array of

[14] http://landcover.usgs.gov/globallandcover.php

[15] https://lpdaac.usgs.gov/dataset_discovery/modis/modis_products_table

[16] www.landcover.org/data/landcover/

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

different gateways,[17] and the so-called collections (processing schemes) have to be selected very carefully, since these differ slightly with respect to spatial and temporal resolution, as well as classification method (Friedl et al., 2010).

The European Space Agency (ESA) also provides a global land cover product. This Climate Change Initiative (CCI) land cover product (also known as GlobCover) was generated from time-series of ENVISAT MERIS images. The dataset covers the majority of the globe (75°N to 56°S, excluding Antarctica) at 300 m spatial resolution (Bartholome and Belward, 2005) and is now available for three time slices representing *c.* 2000, 2005, and 2010.[18] The classification follows the 22-class levels of the Land Cover Classification System (LCCS).

Other land cover products are based on much higher resolution sensor data such as Landsat. One example is the National Land Cover Database (NLCD) (Homer et al., 2012), a Landsat-based, 30 m resolution land cover database for the USA. NLCD provides land surface information such as thematic class, percent impervious surface, or percent tree canopy cover. NLCD data are available for several time steps (1992, 2001, 2006, and 2011) and are free for download from the MRLC website.[19] Also the Gap Analysis Program (GAP) methodology (Scott et al., 1993) has produced a large series of state-wise or regional land cover products (e.g. Lowry et al., 2007), which are very detailed and often used for ecological applications, especially in biodiversity assessment and animal conservation planning (Scott et al., 2001; Pearlstine et al., 2002). The criticism that conservation decisions for animals should not rely on land cover data alone has been raised since error rates are considered too high (Schlossberg and King, 2009). However, in many instances, they are likely to provide important information on local-scale filtering between climatically suitable areas, because of their structural and contextual relevance. Another product often used in Europe is the Coordinated Information on the European Environment (CORINE) land cover (also known as CLC).[20] This combined land cover/land-use dataset is built on a hierarchical legend, which distinguishes >50 classes at finest scales. It comes in two spatial resolutions, 100 m and 250 m. Three datasets exist, one for 1990, one for 2000, and one for 2006, the latter

[17] https://lpdaac.usgs.gov/data_access

[18] http://due.esrin.esa.int/page_globcover.php

[19] www.mrlc.gov/

[20] www.eea.europa.eu/data-and-maps/

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

covering more countries of the European Union than the first. There is also a "change product" available, where land cover/use change has been explicitly mapped.

A slightly different group of interesting land cover datasets is available from what are termed land cover continuous fields (also called fractional cover or sub-pixel classification; see also the NLCD dataset above). These maps usually originate from coarse resolution satellite images (MODIS, AVHRR, etc.) where associating a single discrete land cover class to a pixel is difficult due to the small-scale structure of land cover in many parts of the world (DeFries et al., 1997, 1999; Hansen et al., 2002; Schwarz and Zimmermann, 2005). On the other hand, such coarser resolution satellite data are usually available at a much higher temporal resolution, which enhances the capacity for classification due to the multi-temporal nature of the data. In order to maximize the usefulness of this spatially coarser resolution data, the pixels are not assigned to a discrete class, but instead the fraction of each class per pixel is mapped (Ju et al., 2003). Strictly speaking, in spectral terms, such cover fractions do not map the fraction of a land cover class, but the fraction of the elements that make this land cover class: e.g. trees, grass, impervious (Hansen et al., 2002; Schwarz and Zimmermann, 2005). Such global data can be downloaded from the MODIS data website (see link above). Another source of information for global continuous field datasets can be obtained from the global land cover facility (GLCF),[21] both for the MODIS-based as well as AVHRR-based products. These datasets are usually available at 250 m (MODIS MOD44B product) and 1 km (AVHRR) spatial resolution. A downscaled version using Landsat images is available as "Landsat Tree Cover" from the same website. It is available at 30 m spatial resolution globally, and specifically improves estimates in agricultural areas (Sexton et al., 2013). However, such datasets can easily be generated for one's own purposes anywhere on the globe with high accuracy (Schwarz and Zimmermann, 2005), and it can also be easily applied directly to other sensors, such as Landsat (Mathys et al., 2009).

Often though, we are interested in land-use rather than in land cover. This is more difficult to map, since it involves interpreting human use of what can objectively be seen from the above, e.g. mapped grassland might be a meadow or a pasture. It might be hard to distinguish between the two, since the difference is the use and not the cover. Furthermore, grasslands can originate from agricultural use, and would naturally revert

[21] www.landcover.org/data/

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

to forest by means of succession if this use is stopped, while other grasslands are permanent because the conditions are not suitable for forests. However, when projecting any model into the future, we should be aware that climate is not changing independently of changes in human land-use. In fact, human actions result in a modification of the radiative forcing, some of which is direct e.g. burning fossil fuels, but another important factor is land-use change. Thus, human activities including land-use change are the cause of global climate change, and therefore future projections of species or biodiversity may suggest that we include the effects of associated land-use change. Some aspects of land-use are included to a greater or lesser extent in most of the land cover products described above. At a global scale, consistent land-use products are most often available at very coarse spatial resolutions, which are generally too coarse to be used effectively in HSMs. We will therefore not discuss these products here. There are regional or local products available, which are suitable for use in HSMs.

6.1.4 Borders, Political Units, and Other Vector Data

There are several data sources available from which users can access and download free spatial vector datasets. One source is the Global Administrative Areas database (GADM)[22] that contains the spatial data of the world's administrative areas, such as countries, and lower level subdivisions, such as provinces. The data are available in several formats such as shapefile, ESRI geodatabase, RData, and Google Earth kmz. Data can be downloaded either by country or for the whole world.

Another source of vector data is Natural Earth,[23] which also includes countries, disputed areas, first-order admin (e.g. departments, states). Data on populated places, urban polygons, parks and protected areas, and water boundaries are also available with different levels of detail.

A source for European-oriented (but also global) datasets is the ESPON database.[24] It provides access to regional, local, urban, neighborhood, and historical data, but also to indicators and tools that can be used for European territorial development and policy formulation at different geographical levels. The data included in the ESPON database is mainly from European institutions such as EUROSTAT and EEA and is aimed at a wide range of users (researchers, policy makers, stakeholders).

[22] http://gadm.org/
[23] www.naturalearthdata.com/
[24] http://database.espon.eu/db2/

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

Historical data can be obtained from the CShapes database.[25] This database contains worldwide historical maps of country boundaries and capitals in the post-World War II period together with dates of when these changes occurred. The CShapes database is available as a shapefile format or as a package for R (`CShapes`) and can be used for a number of spatial analyses.

Finally, a very useful database for marine information is Marine Regions.[26] It is an integration of the VLIMAR Gazetteer and the VLIZ Maritime Boundaries Geodatabase and includes marine boundaries, fishing zones, ecological classifications of marine ecosystems and names such as seas, sandbanks, seamounts, and ridges. The geographic cover of the database is global.

6.2 Performing Simple GIS Analyses in R

In this chapter, spatial data visualization, processing, and analysis are introduced using functions and packages available in R. Each reader may have their own favorites and can combine these with the functions and commands presented here. We will primarily use regional to global scale datasets that are freely available on the internet. This chapter primarily deals with raster data, which are often used in spatial habitat suitability modeling. The reader learns how to adjust resolution and extent of raster layers, how to re-project and recalculate them, or how to stack and overlay them with point observation data (field samples). Other sections introduce how to generate contours from raster layers or how to use lines or polygons in combination with raster data. In a final section, the generation of a consistent data structure for habitat suitability modeling is introduced.

6.2.1 Introduction

We will make use of the GIS layers available in ESRIs grid and vector formats as well as GeoTIFF, which are the most widely distributed formats for interacting statistics with GIS. In order to prepare for further statistical analyses, simple spatial operations and analyses are introduced, such as building spatial datasets, importing grids, vectors and points, developing new and partly DEM-derived raster layers, intersecting points with grids, or building simple SDMs and predicting the

[25] http://nils.weidmann.ws/projects/cshapes

[26] www.marineregions.org/about.php

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

modeled habitat suitability and associated spatial uncertainties for the whole spatial study area.

In order to carry out these analyses in R, several spatial R packages need to be installed and loaded. Since some of these packages are still under development, it is important to use the right version. The required packages are: `maptools`, `sp`, `rgdal`, and `raster`. Installing the packages is generally comparatively straightforward. Once these packages are installed, they can be loaded as follows:

```
> library(sp)
> library(rgdal)
> library(raster)
> library(dismo)
> library(maptools)
```

Furthermore, and for all future analyses, it is important to make sure that R is directed to the correct working directory:

```
> setwd("PATH/data/")
```

All data loaded in subsequent examples are indicated in the R commands relative to this path.

In addition to the preparation in R, all the required spatial data have to be prepared and are stored, preferably in one major folder. Preparations ideally include converting all raster and vector layers to a common projection and map extent/resolution (for grids). As raster layers can cause memory limitation problems, they should be read as integer grids where possible. Preparation thus may also include a conversion to integers. Floating grids are often best multiplied by a factor of 10 or 100 prior to being converted to integers in order to keep thematic resolution. Most GIS and RS data used in subsequent analyses are available and loaded directly from online sources, while some are available from this book's website.[27] Please download and unpack the data from the book's website locally on your computer. The data are generally stored in subfolders according to their nature and thematic similarity.

6.2.2 Loading the Data and Initial Exploration

Loading data into R using the raster package commands can be done very easily with the `raster()` command. Load those grids you want to use later in your exercises. Here we provide the commands for four

[27] www.unil.ch/hsdm

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

layers, namely: temperature isothermality (bio3), temperature annual range (bio7), mean temperature of coldest quarter (bio11) and annual precipitation (bio12). The data is loaded from the basic subfolder termed "raster" within the set path (see above).

```
> bio3    <-raster("raster/bioclim/current/grd/bio3")
> bio7    <-raster("raster/bioclim/current/grd/bio7")
> bio11   <-raster("raster/bioclim/current/grd/bio11")
> bio12   <-raster("raster/bioclim/current/grd/bio12")
```

These grids are now loaded into R and we can access them directly. In order to evaluate the imported datasets, we can use GIS-type commands that give us this information, similar to the "describe" command in ArcInfo:

```
> bbox(bio7); ncol(bio7); nrow(bio7) ; res(bio7)
           min        max
s1  -180.00000  180.00000
s2   -57.49999   83.50001
 [1] 240
 [1] 94
 [1] 1.5 1.5
```

Wordclim data have been loaded at 1.5° lat/lon resolution, which roughly corresponds to a 167 km spatial resolution at the equator.

We then load the GTOPO30 global DEM. As seen in Section 6.1.1, this dataset is available at 30 arc seconds, translating into roughly 1.85 km at the equator. We also assign a projection to this raster, without much explanation here as this topic is treated later (Section 6.2.8):

```
> elev <- raster("raster/topo/GTOPO30.tif")
> projection(elev) <- "+proj=longlat +datum=WGS84 +ellps=WGS84
    +towgs84=0,0,0"
```

If we now compare the elev and the bioclim grids (e.g. bio7), it becomes obvious that they do not have the same spatial extent, pixel resolution or number of rows and columns:

```
> bbox(elev); ncol(elev); nrow(elev); res(elev)
    min       max
s1 -180 180.00002
s2  -60  90.00001
 [1] 43200
 [1] 18000
 [1] 0.008333334 0.008333334
```

Specifically, we see that `elev` has many more rows and columns than bio7, and also the lower-left coordinate, the extent, and the pixel size differ. They have neither the same resolution nor extent.

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

6.2.3 Resampling, Spatial Alignment, and Indices

In order to generate a clean data structure for modeling and combining all grids into one stack of grid layers for our study area (this facilitates subsequent analyses) we first resample all grids with deviating resolution and extent (here `elev`) with reference to a master grid (here `bio7`). In this way, we resample the `elev` grid from 30 arc second resolution to a 1.5 ° resolution, and align the new `elev` grid to the lower-left corner of the `bio7` grid, which serves as a master for this operation. Two methods are available within the `resample()` command to adjust the resolution, namely nearest neighbor (`method="ngb"`) and bilinear (`method="bilinear"`). The first is fast and picks the value at the central location of the new grid from the old grid with no interpolation. This method is usually recommended for categorical data. The bilinear method is much slower, as it interpolates from all neighboring cells to calculate the new cell values. It is usually preferred for continuous data. For the purposes of simplicity, here this resampling is carried out using the nearest neighbor method (`method="ngb"`).

```
> elev1<-resample(elev,bio7,method="ngb")
```

In order to accurately upscale continuous grids to much larger pixel sizes it is advisable to use the `aggregate()` function, which extends the size of the pixel by a multiple of the original pixel size. This method is much faster than bilinear resampling, although it still interpolates among the aggregated pixels, e.g. by using the mean function (`fun=mean`) as argument. This is done by first aggregating to a resolution close to 1.5°, if this value cannot be obtained by a direct multiple of the original cell, and then interpolating to the exact resolution and extent using the `resample()` command.

Now all raster layers are converted to the same spatial structure and can be visualized. For the plotting, the `terrain.colors()` option represents the default color theme in the raster package. The other three standard themes, `topo.colors()`, `rainbow()` and `heat.colors()`, can also be used.

```
> plot(bio12,main="Bio.12")
> par(mfrow=c(3,1))
> plot(elev1,col=rev(topo.colors(50)),main="Elevation")
> plot(bio3,col=heat.colors(100),main="Bio.3")
> plot(bio11,col=rainbow(100),main="Bio.11")
> par(mfrow=c(1,1))
```

This creates three maps with conserved x- and y-axis ratios. In a regular plot, such axis ratio conservation can be achieved using the option

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

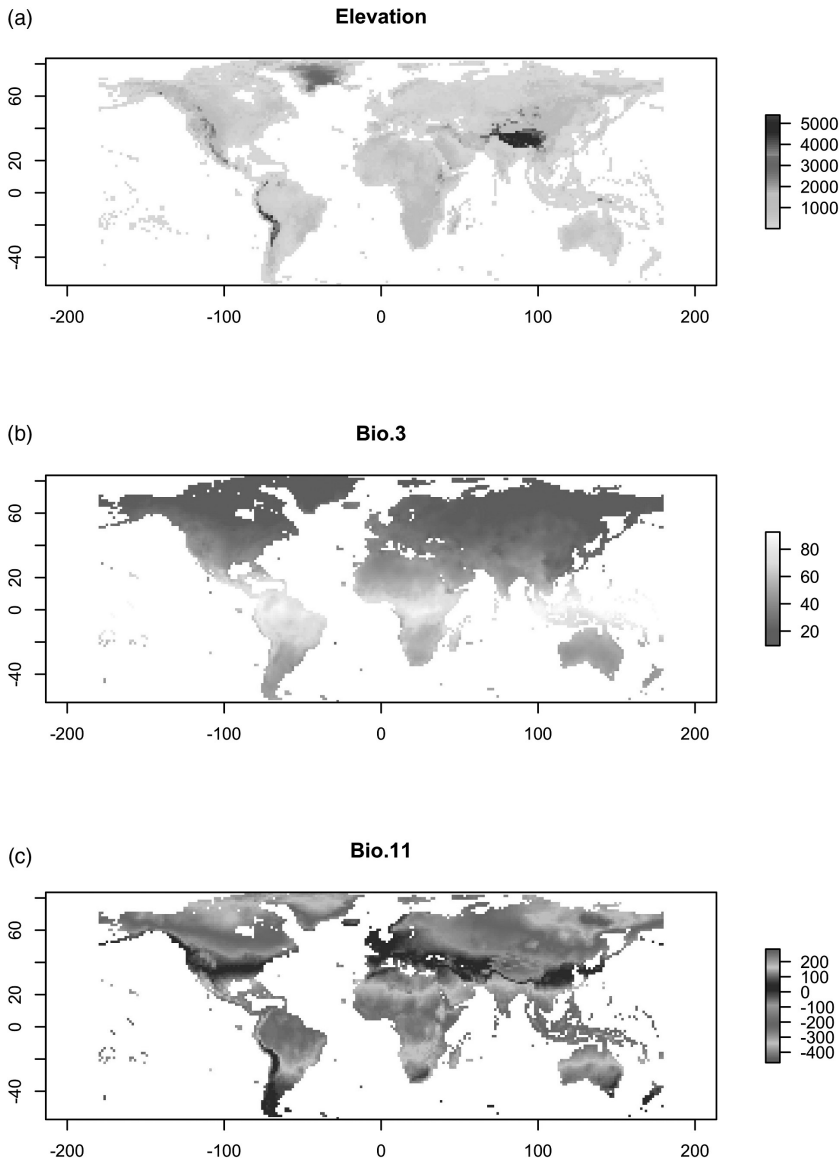


Figure 6.1 Illustration of three maps using the raster package: (a) elev1 = altitude, (b) bio3 = isothermality (×100), and (c) bio11 = mean temperature of the coldest quarter (×10). (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

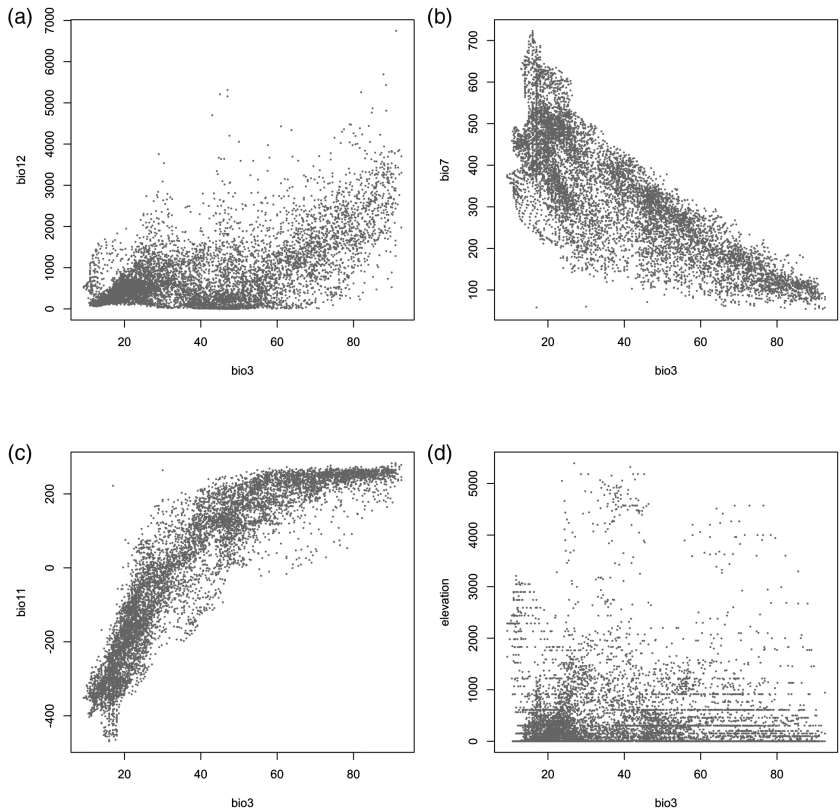


Figure 6.2 Illustration of the pairwise correlation structure of three bioclim and one elevation grid.

`(asp=1)`, but in raster this is not necessary. In the last sequence of commands we have used three different color schemes in one plot window by using the `par()` command (Figure 6.1), which is a very flexible instrument to define plot parameters.

The raster package provides an easy way of observing the correlation between rasters. The same `plot()` command in `raster` can be used to visualize the shape of correlations among raster layers (Figure 6.2):

```
> par(mfrow=c(2,2))
> plot(bio3,bio12,xlab="bio3",ylab="bio12",col="gray55")
> plot(bio3,bio7,xlab="bio3",ylab="bio7",col="gray55")
> plot(bio3,bio11,xlab="bio3",ylab="bio11",col="gray55")
> plot(bio3,elev1,xlab="bio3",ylab="elevation",col="gray55")
> par(mfrow=c(1,1))
```

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

6.2.4 Working with Contours and Lines

Other types of spatial data can also be loaded in R for habitat suitability modeling, namely shapefiles that represent points, lines and polygons. Several packages are useful for reading and writing shapefiles, but some of them require detailed knowledge to do so. We prefer to use the simple `shapefile()` command from the `raster` package, which can be used to read points, lines and polygons. Often, however, we generate lines or points in R, and then we may want to store these (especially lines and polygons) as shapefiles. Here, we use the `rgdal` package to do so.

First, we return to the `elev1` raster object, which we have created. The `contour()` function can then be used to represent selected elevation (Figure 6.3).

```
> plot(elev1)
> contour(elev, nlevels=7, levels=c(0, 500, 1000, 1500, 2000,
3000, 4000, 5000), add=T, labels="", lwd=.2)
```

Of course, the contour function can also be used to generate probability isolines from SDM output rasters, or other isolines from any numerical or continuous raster surface. It is thus a very practical command for adding lines to a raster to better visualize the results.

In order to visualize these contours in more detail, we select South America to plot these contours. We do so by cropping the global elevation model to a smaller extent and then plotting the same contours to the graph as done above, but for the `elev_sa` elevation raster (Figure 6.4).

```
> elev_sa<-crop(elev, extent(-85,-30,-60,15))
> elev_na<-crop(elev, extent(-188,-50,15,90))
> plot(elev_sa, main="Elevation Contours South America")
> contour(elev_sa, nlevels=7,
levels=c(0, 500, 1000, 1500, 2000, 3000, 4000, 5000),
add=T, labels="", lwd=0.2)
```

Next, we need to store the global contour line as a spatial object, and finally write it into a line shapefile. To do this, we use the `rasterToContour()` command in the `raster` package.

```
> iso<-rasterToContour(elev, nlevels=7, levels=c(0, 500, 1000,
1500, 2000, 3000, 4000, 5000))
> writeOGR(iso, dsn="vector/globe", layer="isolines",
"ESRI Shapefile", check_exists=T, overwrite_layer=T)
```

Essentially, the command for creating contour lines is very similar to drawing them on an existing plot. We then store the lines as shapefiles using the `writeOGR()` command from the `rgdal` package. We now

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

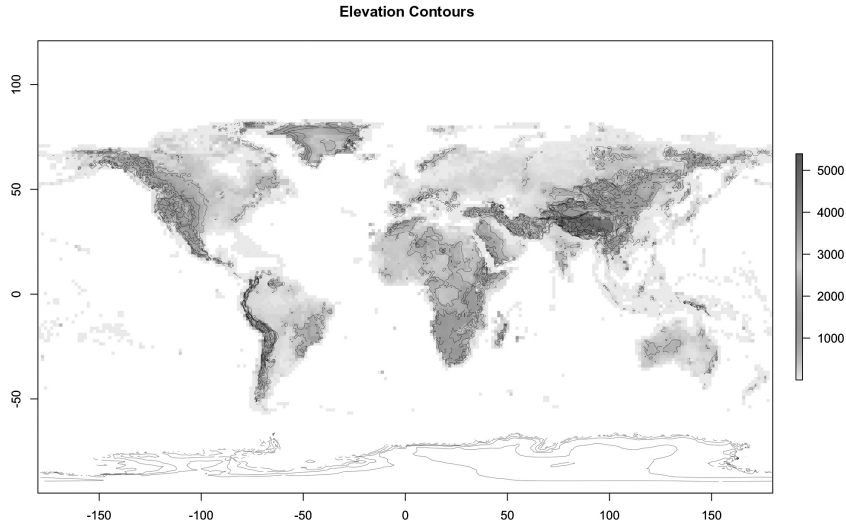


Figure 6.3 Elevation contours displayed over digital elevation model. (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

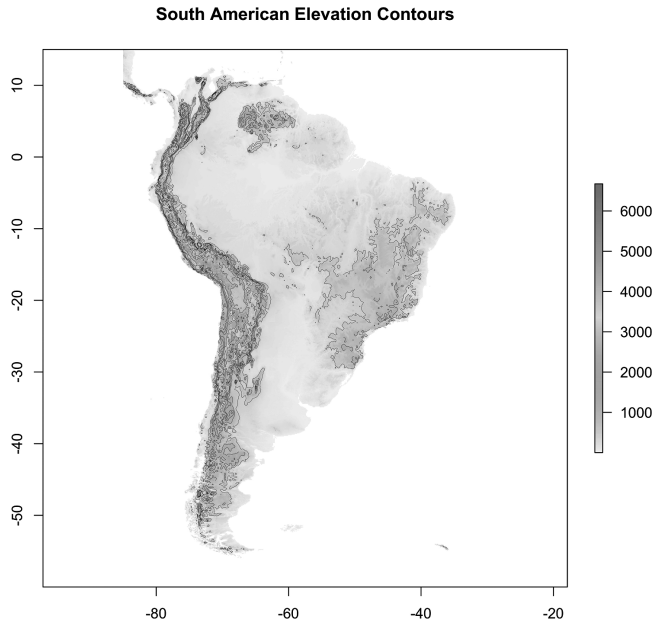


Figure 6.4 Contours displayed over a digital elevation model of South America. (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

can read this (or other) shapefiles using the simple `shapefile()` command from the raster package.

```
> iso1<- shapefile("vector/globe/isolines")
> plot(elev_na,col=terrain.colors(40),main="Elevation Contours")
> lines(iso1, lwd=0.2)
```

6.2.5 Raster Analyses of Type "Global"

Here, we calculate simple raster operations. These are called "global" operations, because each cell is treated exactly the same way, meaning that the operation is applied equally to all cells. In the first example, we assume that temperature on the one hand is related to elevation, latitude and longitude. If we conduct this type of analysis using ordinary least-square regressions, we might obtain the following output:

```
> summary(reg.lm)

Call: lm(formula = bio11 ~ elev + abs(lat) + abs(lon) + I(lon^2)

Coefficients:
              Estimate   Std. Error t value Pr(>|t|)
 (Intercept)  4.138e+02   2.146e+00  192.82   <2e-16 ***
alt          -3.624e-02   8.603e-04  -42.13   <2e-16 ***
abs(lat)     -8.216e+00   2.885e-02 -284.82   <2e-16 ***
abs(lon)     -1.794e+00   5.389e-02  -33.29   <2e-16 ***
I(lon^2)      7.122e-03   3.206e-04   22.22   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 58.56 on 8290 degrees of freedom
Multiple R-squared:  0.9191,  Adjusted R-squared:  0.9191
F-statistic: 2.356e+04 on 4 and 8290 DF,  p-value: < 2.2e-16
```

We see that we can express the mean temperature of the coldest quarter (bio11) from a sample of pixels in the climate map as a function of elevation, latitude, and longitude. We now apply this function to each cell of the elevation grid and then plot the resulting grid (Figure 6.5). To implement this calculation, we first need to load the two missing raster files (lat, lon) from TIFF files and then calculate the difference between the observed and the modeled mean temperature of the coldest quarter:

```
> lat<-raster("raster/other/latitude.tif")
> lon<-raster("raster/other/longitude.tif")
> tcold<-4.138e+02 + (-3.624e-02 * elev1) + (-8.216e+00 *
abs(lat)) + (-1.794e+00 * abs(lon)) + (7.122e-03 * lon^2)
> diff_obs_model_temp <-bio11 - tcold
> par(mfrow=c(2,1))
> plot(tcold, col=rev(rainbow(100))[20:100],main="Modelled mean
temperature of the coldest quarter")
> contour(elev, nlevels=7,
levels=c(0, 500, 1000, 1500, 2000, 3000, 4000, 5000), add=T,
labels="", lwd=.3)
```

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

```
> plot(diff_obs_model_temp, col=rev(rainbow(100))[20:100],main=
"Difference between modelled and observed temperatures")
> contour(elev, nlevels=7, levels=c(0, 500, 1000, 1500, 2000,
3000, 4000, 5000), add=T, labels="", lwd=.3)
> par(mfrow=c(1,1))
```

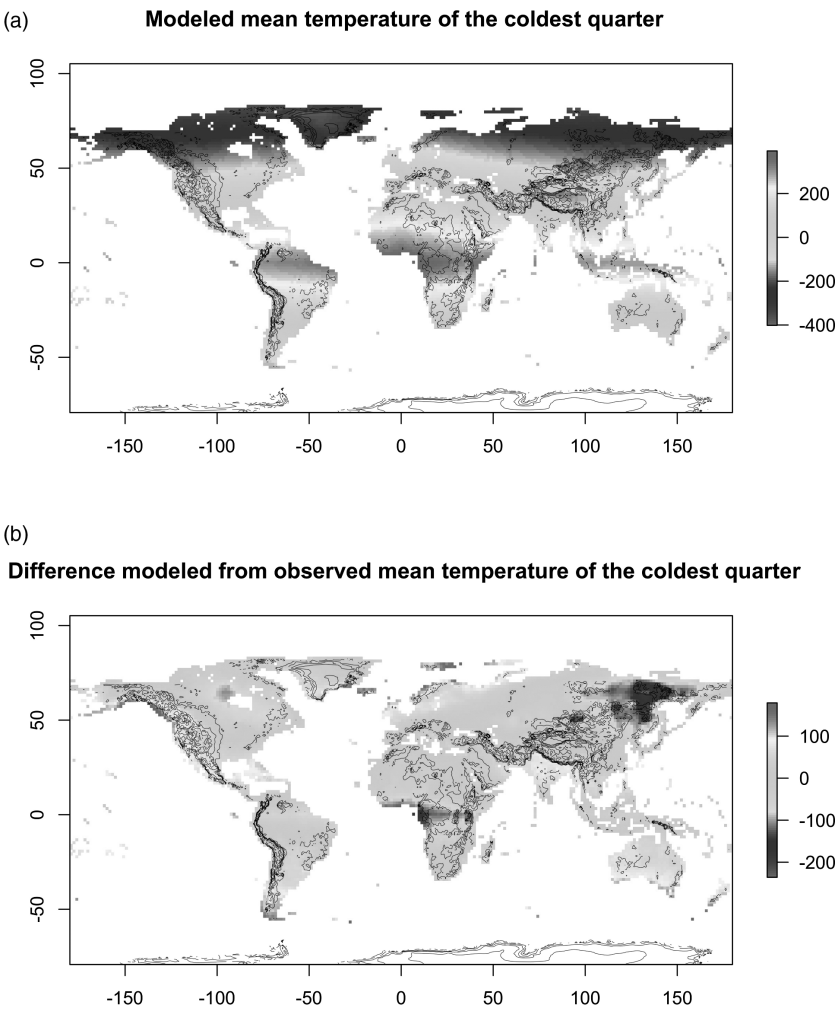


Figure 6.5 Elevation contours displayed over (a) modeled mean temperature of the coldest quarter and (b) difference between modeled and observed temperatures. (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

In the same way, we can perform any mathematical operation between rasters or between rasters and scalars, so that each cell is processed identically over the whole raster extent.

6.2.6 Raster Analyses of Type "Focal" and Terrain Analyses

The next typical raster operation is the "focal" type. We talk about focal analyses when applying neighborhood-based analyses. A moving window is usually run across the whole grid, a function is applied to the cells within the window, and the result of this function is written in the center cell of the window. The window is then moved from one cell to the next row/column-wise from upper left to lower right. In the following example, we calculate the difference between the elevation grid and the average elevation in a moving window of 15 × 15 cells. The calculation of the average elevation in a window (i.e. the focal analysis) means to apply a smoother to the elevation, and we compare this smoothed elevation to the original elevation. Therefore, if the center cell now has a "higher" value than the surrounding 15 × 15 cells, then this cell must be a positioned on a ridge or peak. Likewise, if the center cell has a considerably lower elevation than the surrounding cells, then it must be positioned in a toe slope or gully. If the difference is negligible, then the center cell of the moving window must be either a flat plain or an even slope. Such analyses can be used to classify the terrain. Here, we carry out this analysis on a *c.* 10 km DEM of South America. In order to do so, we first crop a *c.* 1 km DEM called `elev` (see above) to the extent of South America and then we aggregate this temporary grid by an aggregation factor of 10.

```
> tmp<-crop(elev, extent(-85,-30,-60,15))
> elev_sa10<-aggregate(tmp, fact=10, fun=mean, expand=TRUE,
na.rm=TRUE)
```

Next, we need to define the focal window including the weights. In this analysis, we give equal weight to all cells in a quadratic 15 × 15 cell window. The window weights matrix is created as follows so that all weights have a value of 1:

```
> w <- matrix(rep(1,225), nr=15, nc=15)
```

Finally, we calculate the focal operation over the 10 km DEM using the `w` window weights and we subtract this focal analysis result from the original elevation grid. We call the resulting raster `TopEx`, representing topographic exposure with negative values representing sinks and valleys and positive values representing ridges and peaks.

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

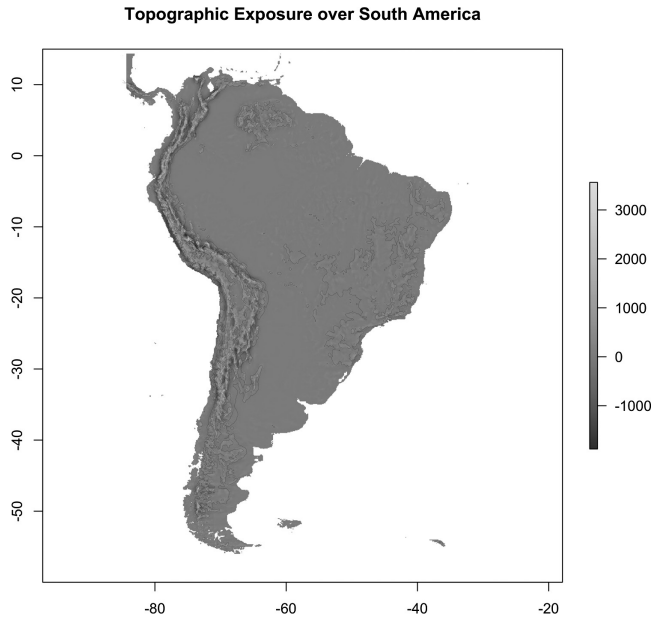


Figure 6.6 Topographic exposure over South America calculated from a focal analysis.

```
> TopEx <- elev_sa10 - focal(elev_sa10, w=w, fun=mean, na.rm=T)
```

To plot this raster, we first define a color scale that is optimal for terrain data, be it solar radiation or topographic exposure data. Then we apply this color scheme to the plot so that positive values are shown in light shades and negative values in dark shades (Figure 6.6).

```
> topography.c  <-colorRampPalette(c("dodgerblue4",
"lemonchiffon", "firebrick3"))
> plot(TopEx,col=topography.c(100),
main="Topographic Exposure over South America")
> contour(elev_sa10, nlevels=7,
levels=c(0, 500, 1000, 1500, 2000, 3000, 4000, 5000),
add=T, labels="", lwd=.1)
```

The default option in the focal analysis requires that values be only calculated for those windows that have all 15 × 15 (= 225) cells available with numerical information. Since that would remove some of the marginal area along the coast, we set `na.rm=T`. This setting removes unavailable or missing values (NAs) from the focal computations and allows calculating an output for windows with fewer than 225 cells with numerical values in our case.

Terrain analyses constitute a special kind of focal analysis. This type of analysis can be used to calculate slope, aspect, or to shade a DEM. Here,

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

we prepare to shade the original *c.* 1 km DEM called `elev` using the `hillShade()` command.

```
> slope <- terrain(elev, opt='slope')
> aspect <- terrain(elev, opt='aspect')
```

These two maps are now used to shade the terrain model `elev`, from which slope and aspect are derived. Here, we set the altitude angle to 30° and the azimuth angle to 315°, in order to illuminate the terrain model.

```
> hillshade <- hillShade(slope, aspect, 30, 315)
```

The hill-shaded raster is stored here as a TIFF file for later use, it can then be read from here again in a new R session.

```
> writeRaster(hillshade, "raster/topo/hillshade.tif",
overwrite=T)
> hillsh<-raster("raster/topo/hillshade.tif")
```

Such maps can now be used as background images for illustrating thematic layers. We illustrate this with the example of North America, first by cropping the global hillshade image to the North America extent, and then by overlaying the elevation map (`elev`) in a semi-transparent manner. In order to do this, we redefine the plot extent as above for North America. The argument `alpha=0.5` adds semi-transparency, making the superimposed layer 50% transparent. In this way, the underlying hillshade is partly visible.

```
> plot_extent<-extent(-124,-66,24,50)
> hillsh_na<-crop(hillshade, extent(-188,-50,15,90))
> plot(hillsh_na, col=grey(0:100/100), legend=FALSE, axes=F,
ext=plot_extent)
> plot(elev_na,col=terrain.colors(100),alpha=.5,add=T,
ext=plot_extent)
```

In the next plotting example, we create our own color palette, mimicking the elevation color ramp of ArcGIS. We first generate a function for this color palette, and then we generate 100 color values.

```
> dem.c<-colorRampPalette(c("aquamarine", "lightgoldenrodyellow",
"lightgoldenrod", "yellow", "burlywood", "burlywood4", "plum",
"seashell"))
> cols<-dem.c(100)
```

Since we cannot assign an `alpha` value in this function, we simply paste the transparency values for 60% transparency to the BinHex color code. For the alpha values 0.0, 0.3, 0.4, 0.5, 0.6, and 1.0, we add the values "00", "4D", "66", "80", "99", "FF", respectively (with alpha = 0.0 being

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

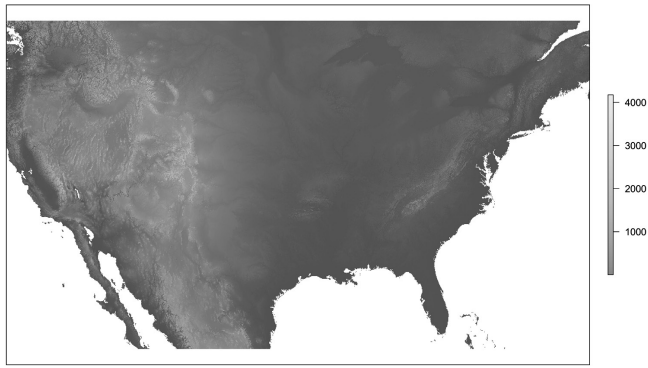


Figure 6.7 Elevation map draped semi-transparently over hill-shaded DEM.

fully transparent and alpha = 1.0 being intransparent). Finally, we plot the map with this color ramp as described above and an alpha value of 0.4 (Figure 6.7). We plot this map again at a smaller extent than the `hillsh_na` raster, using the same extent as above.

```
> cols<-paste(cols,"66",sep="",collate="")
> plot(hillsh_na, col=grey(0:255/255), legend=FALSE, axes=F,
    ext=plot_extent)
> plot(elev_na, col=cols,add=T, ext=plot_extent)
```

There are numerous other possible analyses. Only a few examples have been provided here. Read for example the manuals for the `raster`, `maptools`, `rgdal`, and `sp` packages in order to find out more. Again, for large datasets, the GIS functionality may be somewhat slow. However, with medium to small datasets, R offers an extremely flexible and powerful way of combining statistical modeling with GIS functions.

6.2.7 Stacking Grids to a Grid Stack

We often prepare environmental data in a raster format (grid) and sample these grids with our observational data points (x/y-coordinates) in order to build an environmental database for our field data. We can do this very easily in R by first stacking all the grids together (`stack()` command from `raster` package) and then sampling the whole stack with one command (`extract()` from `raster` package):

```
> world.stk <- stack(elev1,bio3,bio7,bio11,bio12)
```

We can easily summarize the information in a grid stack by typing the `summary` command: `summary(world.stk)`

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

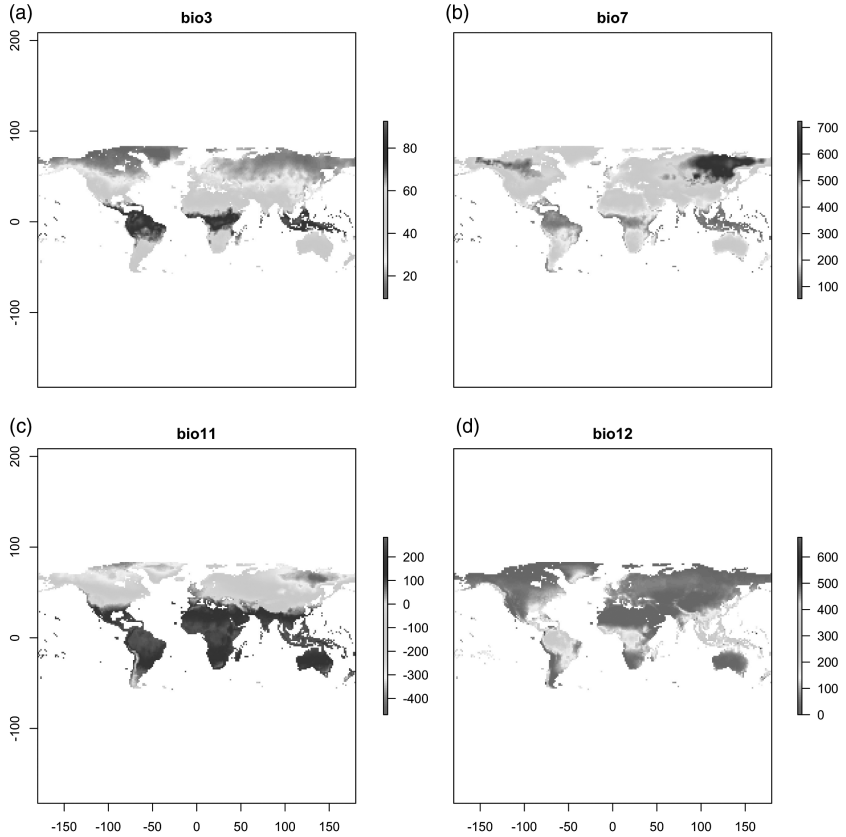


Figure 6.8 Visualization of all elements in a stacked grid. (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

Single grids from a grid stack can be selected using the `$` selection (e.g. `world.stk$bio7`). In addition, we can obtain a very rapid overview of a stack by mapping its content (Figure 6.8). This is only recommended if the stack contains a small number of grids. Here, we plot the stack for layers 2 to 5, thus not plotting the first layer, i.e. elevation:

```
> plot(world.stk[[2:5]], col=rainbow(100,start=.0,end=.8))
```

Working with grid stacks has several advantages. It enables sampling all elements of a stack from a point file using a single command (see below), or one can process all stacked grids equally with one single command. One example of this latter option is changing the cell size (spatial resolution) of all grids in our stack. Assuming we want to convert our 1.5° stack of rasters (our `world.stk`) to a lower resolution (3 × 3 degree),

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

we need to aggregate the rasters by a factor of 2 (storing the mean of four cells into larger 3° cells).

```
> world.stk3 <- aggregate(world.stk,fact=2, method="mean")
```

When plotting this stack, we can immediately see that the cells are larger than in the original stack. This represents an upscaling procedure (with respect to grain size).

6.2.8 Re-Projecting Grids

Rasters (or vector type files) assembled from various sources might not always be found in the same projection. Overlaying these environmental files with a set of sample points for modeling purposes or creating a stack of raster files requires all the files to be part of the same projection. We have seen previously how to assign a projection. Here we work through an example of the steps that need to be taken when reading a raster from a different projection.

Here, we read long-term (30-year normal) annual climate data for precipitation and temperature from the PRISM project (see Section 6.2.1).

```
> prec_yearly_usa <- raster("raster/prism/prec_30yr_normal_
annual.asc")
> tave_yearly_usa <- raster("raster/prism/tave_30yr_normal_
annual.asc")
```

If we check extent and projection, then we realize: first, that the extent has coordinates similar to the lon/lat WGS84 projection and second, that the projection is already set, but that the datum is different:

```
> extent(prec_yearly_usa)
> projection(prec_yearly_usa)
 [1] "+proj=longlat +datum=NAD83 +no_defs +ellps=GRS80
+towgs84=0,0,0"
```

The datum and the ellipsoid of the projection differ from the ones used when reading the Worldclim data (bio3, etc.). In order to superimpose these maps correctly with the points, vectors and rasters that use the WGS84 datum and ellipsoid, we need to re-project the PRISM rasters to the same WGS84 datum and ellipsoid. It is also interesting to see that the raster already has the projection information, despite being read from an ascii file. This information is attached, because in the same folder of PRISM climate files, there are also associated `*.prj` files available with the same name as the `*.asc` ascii files. The `raster` environment recognizes these files, and reads the projection information from them. Removing these `*.prj` files from the directory results in reading un-projected raster files.

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

In the following example, we re-project the two rasters to a lon/lat WGS84 projection, and in doing so, we adjust the resolution to 0.025°. We could also keep the original resolution of 0.00833333°, yet this would take considerably longer.

```
> prec_yearly_usa_wgs <- projectRaster(prec_yearly_usa,
res=0.025, crs="+proj=longlat +datum=WGS84 +no_defs +ellps=WGS84
+towgs84=0,0,0", method="bilinear")
> tave_yearly_usa_wgs <- projectRaster(tave_yearly_usa,
res=0.025, crs="+proj=longlat +datum=WGS84 +no_defs +ellps=WGS84
+towgs84=0,0,0", method="bilinear")
```

The new output raster now has a 0.025° spatial raster resolution. This translates to *c.* 2.5 km in a metric projection. The original 0.00833333° resolution would project to *c.* 800m cell size if projected to a metric projection such as Albers equal area (aea) or Lambert azimuthal equal area (lazea). It is important to note that the lon/lat geographic coordinate system does not conserve area or angles, and therefore cannot be used to perform any area- or distance-based calculations. For such analyses, all layers should be projected to equal area-based projections, for example. An overview of PROJ4 projections that represents the basis for the projection definitions in R are available online.[28]

6.2.9 Importing and Overlaying Species Data

One frequently used GIS analysis is to import and overlay field sampled species distribution data with environmental predictor layers to later model their habitat suitability (Part III onward). Here, we import a set of distribution data for *Pinus edulis* L., downloaded from GBIF (see Section 6.1.1) in July 2014.

```
> pinus_edulis<-read.table("tabular/species/pinus_edulis.txt",
h=TRUE,sep=",")
```

Such a file is read as a data frame object, which includes two columns that represent coordinates (lat and lon). However, the file is not of class `spatial`. We can check this with the `class()` command.

```
> class(pinus_edulis)
```

Next, we define the object to be of class `spatial`, and therefore we need to assign which columns represent the x- and the y-coordinates.

```
> coordinates(pinus_edulis) <- c("lon", "lat")
```

[28] http://geotiff.maptools.org/proj_list/

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

When comparing the `pinus_edulis` object with the original version, there are no longer any coordinates visible in this object. These are stored separately and can be retrieved using the following command:

```
> coordinates(pinus_edulis)
```

When checking again using the `class()` command, the object is now of class `spatial`. However, we also see that no coordinate system has yet been defined for this spatial object. So we then need to define it:

```
> projection(pinus_edulis) <- "+proj=longlat +datum=WGS84 +no_
defs +ellps=WGS84 +towgs84=0,0,0"
```

This finalizes the transformation of a simple data frame object into a spatial object. Such objects require both the columns that represent the coordinates, and a projection to be defined. Numerous datasets are available in the simple Lon/Lat geographic projection, which is a simple coordinate system. More advanced projections are needed to ensure that either the angles or areas are correctly represented.

As an alternative to reading the *Pinus edulis* data from a file, we can derive these distribution points directly from online sources such as the Global Biodiversity Information Facility (GBIF). This can be done from within R using the `dismo` package.

```
> library(dismo)
> pinus_edulis <- gbif('pinus', 'edulis', download=T,
geo=T, sp=T, removeZeros=T)
```

Using the option `(sp=T)`, we ensure that the generated file is converted directly into an object of class `spatial`, and not only into a tabular data frame object. The new spatial object has just one column (next to the invisible coordinates), namely the download date. We just give it this name:

```
> names(pinus_edulis)[1]<-"dwnld.date"
```

If we want to download more variables, we need to set respective arguments. We can now save this spatial data object in a text file for later use. We do this by converting it to a comma-separated CSV file.

```
> write.table(data.frame(pinus_edulis@coords, pinus_edulis@data),
"tabular/species/p_edulis.txt", sep="," ,row.names=FALSE)
```

An almost identical method would be to use the `write.csv()` command. This directly generates comma-separated files, but requires more care when reading the data back if using commands other than `read.csv()`, such as the `read.table()` command, for example. Both methods are very similar, but attention must be paid to the way row names are written and imported.

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

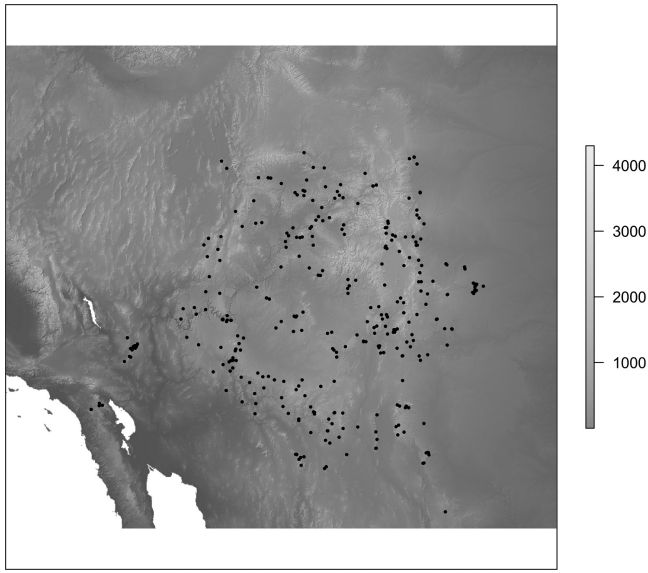


Figure 6.9 Downloaded distribution points of *Pinus edulis* from GBIF.

```
> write.csv(data.frame(pinus_edulis@coords, pinus_edulis@data),
"tabular/species/p_edulis.csv")
```

Next, we want to extract climate data from the `world.stk` object used above (Section 6.2.7) so that we overlay several stacked, bioclimatic raster objects with one single command using the `bilinear` interpolation method for a set of points.

```
> pts.clim<-extract(world.stk, pinus_edulis, method="bilinear")
> pin_edu.clim<-data.frame(cbind(coordinates(pinus_edulis),
pts.clim, pinus_edulis@data))
```

The `extract()` command generates a data frame object (`pts.clim`) containing climate data, which is merged with the coordinates (`coordinates(pinus_edulis)`) and the data (`pinus_edulis@data`) from the `pinus_edulis` spatial object. The resulting data frame is now no longer a spatial object, but we can easily generate such a structure again:

```
> coordinates(pin_edu.clim)<-c("lon","lat")
```

Here, we now plot this file over North America in order to visualize the presence of *Pinus edulis* as downloaded from GBIF using a predefined extent that is smaller than the hillshade created above (Figure 6.9, note that this figure is printed in gray).

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

```
> map.ext<-extent(-120,-100,30,44)
> plot(hillsh_na, col=grey(0:100/100), legend=FALSE, axes=F,
ext=map.ext)
> plot(elev_na,col=cols,add=T,ext=map.ext)
> plot(pinus_edulis, pch=16, cex=.5, add=T)
```

6.2.10 Generating a Uniform Spatial Data Structure for Modeling and Analysis

We demonstrate how a simple data structure is prepared for species distribution modeling and how we can project a simple model using the GIS functionality in R. The whole of Part III will be devoted to deepening our understanding of many different statistical modeling approaches, while Part V is meant to introduce the projections in space and time in more detail. Here, we generate a first simple data structure describing the distribution of the red fox (*Vulpes vulpes*), one of the species used later in the examples in Part III. A data structure is imported that contains a training or calibration (`cal.txt`) and a validation or evaluation (`eva.txt`) dataset, both created by regularly sampling the global range of the species. We then overlay these datasets with four bioclim variables from the `world.stk` raster stack, and fit a very simple GLM, see Chapter 10). We then simply plot this model for the purposes of illustration, but we do not test it against the evaluation dataset (which will be treated in Part IV), and we do not further explore the spatial (or temporal) projections (which will be covered in Part V).

In our example, we use *Vulpes vulpes* and four bioclimatic predictors to build a simple GLM-based distribution model. We process the model from a simple data frame and chose not to transform this data frame into an object of type `spatial`. We also perform a stepwise variable reduction and then use the function `ecospat.adj.D2.glm()` from the `ecospat` library to calculate the adjusted D^2 values (`adj.D2`) (see Guisan and Zimmermann, 2000 for details).

```
> pts.cal<-read.table("tabular/species/cal.txt")
> pts.eva<-read.table("tabular/species/eva.txt")
> plot(pts.cal[which(pts.cal$VulpesVulpes==0),1:2],pch=15,cex=.3,
col="grey50",xlab="Longitude",ylab="Latitude")
> points(pts.eva[which(pts.eva$VulpesVulpes==0),1:2],pch=15,
cex=.3, col="grey85")
> points(pts.cal[which(pts.cal$VulpesVulpes==1),1:2],pch=16,
cex=.4, col="firebrick3")
> points(pts.eva[which(pts.eva$VulpesVulpes==1),1:2],pch=16,
cex=.4, col="seagreen3")
```

In this way, the calibration and the evaluation points are plotted in maroon and sea green, respectively, for presence points (Figure 6.10). The absence points for the two datasets are presented in darker and lighter shades of gray.

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

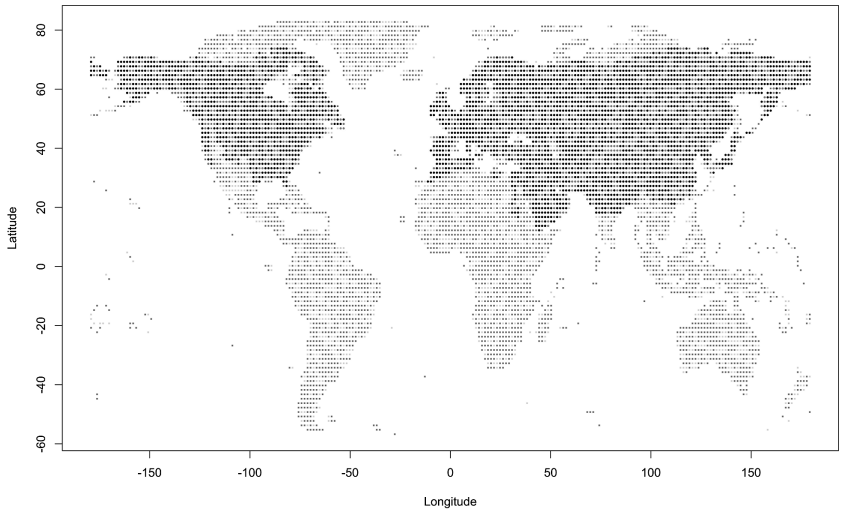


Figure 6.10 Distribution of presence and absence points sampled from a global range map of *Vulpes vulpes*. Presence calibration (red) and evaluation (green) points reflect the species' range, while the absence points of the calibration (darker gray) and the evaluation (lighter gray) datasets reflect areas of absence for the species. (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

We then overlay the calibration and evaluation points with the `world.stk` object using the `extract()` function from the `raster` package. Here, we concatenate two commands. First, we extract the four grid layers from the `world.stk` raster stack using the `extract()` command (layers 2 to 5). This simply generates a matrix of bioclim variables for each of the coordinates from the `pts.cal` object, the coordinates for the extraction are found in the first two columns. Second, we combine column eight from the original calibration and evaluation data frames with the extracted bioclim variables, so that these are joined at the end. This eighth column contains the distribution data for *V. vulpes*. By combining the overlaid matrices with the original data frames, we generate two new data frames:

```
> pts.cal.ovl<-cbind(pts.cal[,8],extract(world.stk[[2:5]],
pts.cal[,1:2]))
> pts.eva.ovl<-cbind(pts.eva[,8],extract(world.stk[[2:5]],
pts.eva[,1:2]))
```

Now, we first need to remove the `NA` values using the `na.omit()` command and at the same time convert the object to a data frame. Second,

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

we need to assign the species (*V. vulpes*) a name in the first column. The first omits points that were sampled from the coarse global distribution map, but were actually located in the sea according to the bioclimatic variables. It is also easier to operate with data frames when modeling species distributions. We therefore apply the `data.frame()` command, which converts the numerical matrix into a data frame.

```
> pts.cal.ovl<-data.frame(na.omit(pts.cal.ovl))
> pts.eva.ovl<-data.frame(na.omit(pts.eva.ovl))
> names(pts.cal.ovl)[1]<-"Vulpes.vulpes"
> names(pts.eva.ovl)[1]<-"Vulpes.vulpes"
```

We have now completed our data preparation and can fit a simple GLM object.

A simple GLM model is fitted to illustrate GIS capability in R. Here, we fit a four-parameter model (bio3, bio7, bio11, and bio12) with both linear and quadratic terms, and we perform simple stepwise bi-directional parameter selection in order to optimize this model.

```
> vulpes.full <- glm(Vulpes.vulpes~bio3+I(bio3^2)+bio7+I(bio7^2)+
bio11+ I(bio11^2)+bio12+I(bio12^2), family="binomial",
data=pts.cal.ovl)
> vulpes.step <- step(vulpes.full, direction="both", trace=F)
```

Next, we load the `ecospat` library to be able to use the `ecospat.adj.D2.glm()` function, which calculates the adjusted D^2 calibration strength of a GLM according to Weisberg's (1980) formula (see Guisan and Zimmermann, 2000).

```
> library(ecospat)
> ecospat.adj.D2.glm(vulpes.full)
 [1] 0.6141926
> ecospat.adj.D2.glm(vulpes.step)
 [1] 0.6142384
```

We find that both models give roughly the same adjusted D^2 value, with a slightly higher value for the stepwise-optimized model. The (unadjusted) D^2 would be higher for the full model, and slightly lower for the stepwise-optimized model. However, the adjusted D^2 considers the number of parameters and the number of observations used, and thus penalizes the stepwise-optimized model, which has the linear term of the bio7 variable removed, less:

```
> summary(vulpes.step)
Call:
glm(formula = Vulpes.vulpes ~ bio3 + I(bio3^2) + I(bio7^2) +
    bio11 + I(bio11^2) + bio12 + I(bio12^2), family = "binomial",
    data = pts.cal.ovl)
```

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

```
Deviance Residuals:
    Min       1Q   Median       3Q      Max
-2.9355  -0.2531   0.0000   0.3655   3.8624

Coefficients:
              Estimate Std. Error  z value  Pr(>|z|)
 (Intercept) -7.596e+00  7.330e-01 -10.362  < 2e-16 ***
bio3          3.711e-01  4.003e-02   9.269  < 2e-16 ***
I(bio3^2)    -6.517e-03  5.446e-04 -11.967  < 2e-16 ***
I(bio7^2)     3.084e-05  1.233e-06  25.009  < 2e-16 ***
bio11         5.277e-03  7.390e-04   7.141 9.25e-13 ***
I(bio11^2)   -2.346e-05  2.582e-06  -9.084  < 2e-16 ***
bio12         2.694e-03  2.776e-04   9.702  < 2e-16 ***
I(bio12^2)   -8.662e-07  1.228e-07  -7.052 1.76e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

 (Dispersion parameter for binomial family taken to be 1)

    Null deviance: 8178.2  on 5899  degrees of freedom
Residual deviance: 3151.1  on 5892  degrees of freedom
AIC: 3167.1

Number of Fisher Scoring iterations: 8
```

Projecting this model to space is now a straightforward process using the GIS functionality in R. We simply predict to the `world.stk` grid stack, which we have used to overlay the calibration points over the climate (predictor) layers. We then generate a map (Figure 6.11) of the projected global distribution of *V. vulpes*, and we add the calibration points that we have used to fit the GLM to this model projection. We take these points from the `pts.cal` object, which has *x*- and *y*- coordinates.

```
> vulpes.map<-predict(world.stk,vulpes.step, type="response")
> plot(vulpes.map, col=rev(heat.colors(10)),
main="Predicted distribution: Vulpes vulpes")
> points(pts.cal[which(pts.cal$VulpesVulpes==1),1:2], pch=15,
cex=.25)
```

We will not further test this model here, as the whole of Part IV is devoted to model evaluation. Many more GIS functions are available in R, which can be used for habitat distribution modeling of species. The basic steps introduced so far should now offer a good basis from which readers can further explore this functionality.

6.3 RS-Based Predictors

In this section, we introduce how to download, load, and visualize RS data and how to carry out simple GIS-like analyses using functions and

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

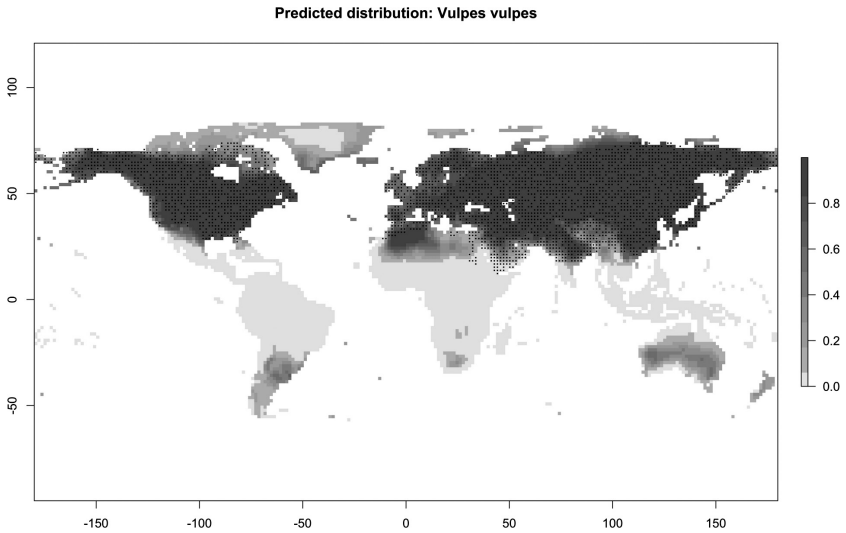


Figure 6.11 Map of the projected global distribution of *Vulpes vulpes* contrasted with the observed distribution points used to calibrate the simple four-parameter GLM model. (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

the available packages in R. Each reader may have his/her own preferred RS datasets, which they can combine with other geospatial data. We will only superficially address the issue of RS data here. Most importantly, this section does not aim to introduce the basics of RS data processing, such as georegistration, relative or absolute atmospheric correction, or cloud masking, for which we refer readers to specialized books (e.g. Normand et al., 2013). Instead, it constitutes a simple introduction to loading and re-calculating remotely sensed data in order to combine these with other GIS layers for powerful statistical analyses (Carlson et al., 2014; Pottier et al., 2014). For this purpose, we use a smaller-scale dataset, which improves the handling and visualization of RS products.

6.3.1 Introduction

RS has a lot to offer biogeographers and macroecologists (Kerr and Ostrovsky, 2003). The careful processing of RS data requires specialist knowledge, and this is not the subject here. However, RS data is increasingly available in pre-processed formats, and can be used like any other form of GIS data, if prepared carefully. One of the biggest advantages of RS data is that it informs us objectively, usually with full coverage of a larger study area, about the state of the Earth's surface at a specific point in time. There are many different systems such as passive optical or active

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

LiDAR and radar RS. Here, we will primarily work with passive optical sensors. Once the dataset has been selected, we have to retrieve the information that best explains the patterns we are interested in.

6.3.2 Data Sources

One of the most widely used data sources for RS imagery is Landsat.[29] The Landsat Program is a series of Earth-observing satellites (EOS) managed by NASA and the US Geological Survey (USGS).[30] The first Landsat satellite was launched in 1972 and the latest satellite, Landsat 8 in February 2013. It continues to provide an excellent RS dataset of almost 40 years by now.

The Landsat 8 satellite collects data for the entire Earth every 16 days with an 8-day difference from Landsat 7. The data collected are available for download free of charge within 24 hours of retrieval.[31]

Landsat data is available as a 4–11 band product, covering the globe with approximately 180 × 180 km single images (called scenes) every 16 days. Most bands have a spatial pixel resolution of roughly 30 × 30 m, depending on latitude. The whole Landsat archive is now available for free, and can be accessed over the internet. A useful archive is the one hosted by the USGS.[32].

```
LandSat8 OLI bands:

    * Band 1      0.43-0.45      Coastal aerosol
    * Band 2      0.45-0.51      Blue
    * Band 3      0.53-0.59      Green
    * Band 4      0.64-0.67      Red
    * Band 5      0.85-0.88      Near IR
    * Band 6      1.57-1.65      SWIR 1
    * Band 7      2.11-2.29      SWIR 2
    * Band 8      0.50-0.68      Panchromatic
    * Band 9      1.36-1.38      Cirrus
    * Band 10     0.52-0.90      Thermal Infrared (TIRS) 1
    * Band 11     0.52-0.90      Thermal Infrared (TIRS) 2

LandSat7 ETM+ bands:

    * Band 1      0.45-0.52      Blue
    * Band 2      0.53-0.61      Green
    * Band 3      0.63-0.69      Red
    * Band 4      0.75-0.90      Near IR
```

[29] http://landsat.usgs.gov/
[30] http://landsat.gsfc.nasa.gov/
[31] http://landsat.usgs.gov/landsat8.php
[32] http://landsat.usgs.gov/band_designations_landsat_satellites.php

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

```
* Band 5      1.55-1.75      Mid-IR
* Band 6      10.4-12.5      Thermal IR
* Band 7      2.09-2.35      Short Wave IR
* Band 8      0.52-0.90      Panchromatic
```

The moderate resolution spectroradiometer (MODIS) provides another frequently used dataset that is also available free of charge. It scans the whole Earth every 1–2 days (depending on latitude) and two satellites – TERRA and AQUA – host the same MODIS sensor. This means a morning and an afternoon scene can be obtained for every day. The sensor has a much larger swath (width of the scanned path) than Landsat, scans a total of 36 different bands and has a pixel resolution of 1 km for 29 bands, except those that mimic Landsat bands, which are available at 500 m resolution (except for the RED and NIR bands that are available at 250 m). One of the big advantages of MODIS is that the data are processed automatically and made available "ready to use," along with a suite of derived variables.

The MERIS sensor is a programmable, medium-spectral resolution, imaging spectrometer on board the ESA's environmental research satellite ENVISAT. MERIS has 15 spectral bands that have a programmable width and location in the visible and near-infrared spectral range (390 nm to 1040 nm). MERIS is a pushbroom instrument that has a 68.5° field of view around the nadir, with a swath width of 1150 km. This allows the instrument to collect data for the entire planet every 3 days in equatorial regions while polar regions are visited more frequently due to the convergence of orbits. The spatial resolution of the data collected by MERIS is 300 m at the nadir (full resolution product) while most common products are generated at 1200 m resolution (reduced resolution product) by aggregating the data to 1200 m.

One of the widest ranging data warehouses is NASA's Reverb, which provides an interface for discovering, accessing, and using EOS data.[33] It allows users to order data from many different sensors and satellite missions. All data holdings stored in the so-called Land Processes Distributed Active Archive Centers (LP DAAC) of the USGS are available here. Data are classified by target application or field, such as atmosphere, cryosphere, land, ocean, and solar. Reverb is probably the best place to order MODIS data, whereas data from the Landsat sensor is best ordered from the Landsat Mission USGS website.[34]

[33] http://reverb.echo.nasa.gov/reverb/

[34] http://landsat.usgs.gov/Landsat_Search_and_Download.php

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

Furthermore, several RS datasets can be downloaded from the ESA's portal,[35] primarily acquired from European satellites like Envisat.

6.3.3 Importing, Resampling, and Grid Stacking

The applicable commands are similar to those used for loading and analysis of GIS data. Here, we assume that all data is available in georegistered and atmospherically corrected TIFF format. We work with bands 1–5 and 7, and leave out the thermal infrared band 6 and the panchromatic band 8, as both are available at different spatial resolutions. We load an image located over north-eastern Switzerland (Path = 194/Row = 027) and we first load a shapefile encompassing the canton of Zurich, in order to crop the image to the shape, so as to reduce the size of the loaded data.

```
> Cantons <-readShapePoly("vector/swiss/Swiss_Cantons.shp")
> Zurich<-Cantons[Cantons$NAME=="ZUERICH",]
```

Next, we load all six Landsat bands.

```
> band1_blue<-raster("raster/landsat/L7_194027_2001_08_24_B10.
TIF")
> band2_green<-raster("raster/landsat/L7_194027_2001_08_24_B20.
TIF")
> band3_red<-raster("raster/landsat/L7_194027_2001_08_24_B30.
TIF")
> band4_nir<-raster("raster/landsat/L7_194027_2001_08_24_B40.
TIF")
> band5_swir1<-raster("raster/landsat/L7_194027_2001_08_24_B50.
TIF")
> band7_swir2<-raster("raster/landsat/L72194027_2001_08_24_B70.
TIF")
```

The next step is to crop the six Landsat bands to the canton of Zurich.

```
> band1_blue_crop<-crop(band1_blue,extent(Zurich))
> band2_green_crop<-crop(band2_green,extent(Zurich))
> band3_red_crop<-crop(band3_red,extent(Zurich))
> band4_nir_crop<-crop(band4_nir,extent(Zurich))
> band5_swir1_crop<-crop(band5_swir1,extent(Zurich))
> band7_swir2_crop<-crop(band7_swir2,extent(Zurich))
```

We then stack all six images into one compound multilayer brick.

```
> L7_010824<-brick(band1_blue_crop, band2_green_crop,
band3_red_crop,band4_nir_crop,
band5_swir1_crop,band7_swir2_crop)
```

Of course, there is a more economical way of executing the same three commands!

[35] https://earth.esa.int/web/guest/data-access

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

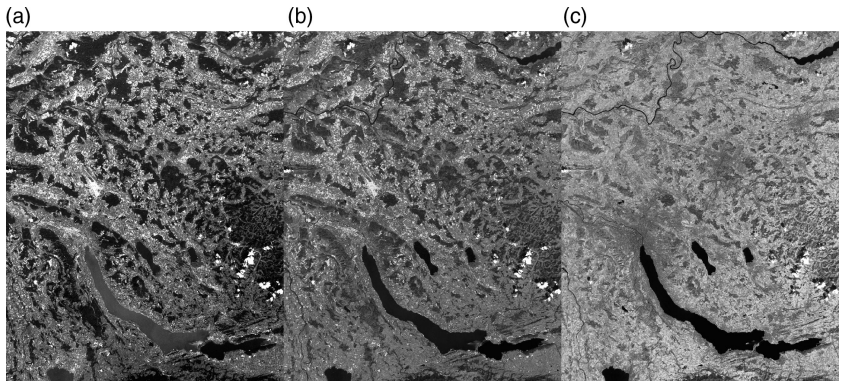


Figure 6.12 Illustration of three-color composites from Landsat bands of the area of Zurich in Switzerland. (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

```
> tmp<-stack(band1_blue,band2_green,
band3_red,band4_nir,band5_swir1, band7_swir2)
> L7_010824<-crop(tmp, extent(Zurich))
```

Finally, we assign layer names and check the naming.

```
> names(L7_010824)<-c("band1_blue","band2_green","band3_red",
"band4_nir","band5_swir1","band7_swir2")
> names(L7_010824)
 [1] "band1_blue" "band2_green" "band3_red" "band4_nir" "band5_
swir1" "band7_swir2"
```

An RGB color composited image from Landsat is usually plotted with bands 3 (red), 2 (green), and 1 (blue), while a false color composite is usually generated from bands 4, 3, and 2. In some cases, bands 7, 5, and 4 (note that band 7 is the sixth band in our stack or brick) are plotted to visualize Landsat scenes. See Figure 6.12 for three examples of representing color composites:

```
> par(mfrow=c(1,3))
> plotRGB(L7_010824,3,2,1,stretch="lin")
> plotRGB(L7_010824,4,3,2,stretch="lin")
> plotRGB(L7_010824,6,4,3,stretch="lin")
> par(mfrow=c(1,1))
```

Currently, there are R packages available that can be used to perform simple to more sophisticated RS analyses. The `landsat` and `RStoolbox` packages have a number of functions for pre-processing and analyzing Landsat, MODIS, ASTER and other remote-sensing data, such as generating most commonly used vegetation indices. Furthermore, other packages such as `MODISTools` and `MODIS` can be used to download, import

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

Table 6.1 *Summary of some of the important indices that are often used in RS-based applications.*

Vegetation index	Equation	Reference
Structural indices		
Normalized difference vegetation index (NDVI)	$NDVI = (R_{NIR} - R_{red})/(R_{NIR} + R_{red})$	Rouse et al. (1974)
Simple ratio index (SR)	$SR = R_{NIR}/R_{red}$	Rouse et al. (1974)
Soil adjusted vegetation index (SAVI)	$SAVI = (1+L)(R_{800} - R_{670})/(R_{800} + R_{670} + L)$ (L = 0.5)	Huete (1988)
Normalized difference water index (NDWI)	$NDWI = (R_{860} - R_{1240})/(R_{860} + R_{1240})$	Gao (1996)
Tasseled cap transformation		Kauth and Thomas (1976)

and process MODIS data directly within R. Finally, other packages like `hyperSpec` or `hsdar`, currently in the early stages of development, can be used for processing hyperspectral RS data.

6.3.4 Data Processing for Ecological Analyses

We will now briefly introduce the recalculation of simple indices, which are often used in RS applications (e.g. Pottier et al., 2014). Once the images are processed, these recalculations correspond to simple GIS-type analyses. We will briefly present the following four indices, namely: the normalized difference vegetation index (NDVI), the simple ratio (SR), the soil adjusted vegetation index (SAVI), and the normalized difference water index (NDWI). Each of these indices enhances certain features, which are not clearly visible in the available bands (see Table 6.1). Therefore, the calculation of such indices is also termed "signal enhancing," since it makes certain signals hidden in the band data accessible to analysts. In addition, we apply the tasseled cap transformation that is designed to analyse and map vegetation and urban development changes detected by various satellite sensor systems. This transformation can be used to monitor crops, analyse and map vegetation for forestry, carbon sequestering and more. However, in order to apply the tasseled cap transformation we need to convert the digital number (DN) values of the downloaded Landsat scene to at-sensor reflectance. We can do this using the `radiocorr()` function in the `landsat` package in R.

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

```
> NDVI <- (L7_010824$band4_nir -L7_010824$band3_red)/(L7_
010824$band4_nir + L7_010824$band3_red)
> NDWI <- (L7_010824$band4_nir -L7_010824$band5_swir1)/(L7_
010824$band4_nir + L7_010824$band5_swir1)
> SR <- L7_010824$band4_nir / L7_010824$band3_red
```

Calculating SAVI and the Tasseled Cap requires At-Sensor or Surface reflectance. In this example, we will calculate At-Sensor reflectance using the `landsat` package in R.

The `landsat` package does not currently accept raster format objects, so we have to convert the data to `SpatialGridDataFrame`.

```
> L7_010824_sp <- as(L7_010824, "SpatialGridDataFrame")
> L7_010824_sp1 <- L7_010824_sp[1]
> L7_010824_sp2 <- L7_010824_sp[2]
> L7_010824_sp3 <- L7_010824_sp[3]
> L7_010824_sp4 <- L7_010824_sp[4]
> L7_010824_sp5 <- L7_010824_sp[5]
> L7_010824_sp7 <- L7_010824_sp[6]
```

Now we can calculate At-Sensor reflectance with the `radiocorr()` function from the `landsat` package using the `apparentreflectance` method. Several parameters are required for the function to run properly, including "gain" and "offset" which are needed to convert the DN to radiance. Most of this information can be found in the metadata provided with the Landsat data or can easily be found using the help associated with the function or in the literature.

```
> library(landsat)
> L7_010824_refl_sp1 <- radiocorr(L7_010824_sp1, Grescale=0.76282,
Brescale=-1.52, sunelev= 48.29, edist=ESdist("2011-08-24"),
Esun=1957, method="apparentreflectance")
> L7_010824_refl_sp2 <- radiocorr(L7_010824_sp2, Grescale=1.44251,
Brescale=-2.84, sunelev= 48.29, edist=ESdist("2011-08-24"),
Esun=1826, method="apparentreflectance")
> L7_010824_refl_sp3 <- radiocorr(L7_010824_sp3, Grescale=1.03988,
Brescale=-1.17, sunelev= 48.29, edist=ESdist("2011-08-24"),
Esun=1554, method="apparentreflectance")
> L7_010824_refl_sp4 <- radiocorr(L7_010824_sp4, Grescale=0.87258,
Brescale=-1.51, sunelev= 48.29, edist=ESdist("2011-08-24"),
Esun=1036, method="apparentreflectance")
> L7_010824_refl_sp5 <- radiocorr(L7_010824_sp5, Grescale=0.11988,
Brescale=-0.37, sunelev= 48.29, edist=ESdist("2011-08-24"),
Esun=215, method="apparentreflectance")
> L7_010824_refl_sp7 <- radiocorr(L7_010824_sp7, Grescale=0.06529,
Brescale=-0.15, sunelev= 48.29, edist=ESdist("2011-08-24"),
Esun=80.67, method="apparentreflectance")
```

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

Now that we have reflectance values (ranging from 0 to 1) we can calculate indices such as SAVI, TSAVI or tasseled cap. Tasseled cap is calculated using the `tasscap()` function from the `landsat` package. Here we choose the option `sat=7` since the satellite data we are using are from the Landsat ETM (Landsat 7) sensor. Finally, we can extract the three newly generated rasters named "brightness," "greenness," and "wetness," which represent band recombinations, and provide ecologically more meaningful information than the raw bands.

```
> L7_010824_tc <- tasscap("L7_010824_refl_sp",  sat = 7)
> L7_010824_Brightness <- raster(L7_010824_tc[[1]])
> L7_010824_Greenness <- raster(L7_010824_tc[[2]])
> L7_010824_Wetness   <- raster(L7_010824_tc[[3]])
```

In order to estimate SAVI we need to define the parameter "`L`" that varies according to the amount of green vegetation or green vegetation cover: in regions with dense vegetation, `L=0` and in areas with no green vegetation, `L=1`. Generally, `L=0.5` works well in most situations and is the default value used.

```
> L <- 0.5
> SAVI <- ((raster(L7_010824_refl_sp4) -
raster(L7_010824_refl_sp3))/(raster(L7_010824_refl_sp4) +
raster(L7_010824_refl_sp3) + L) )* (1+L)
```

Finally, we plot the results of our simple analyses using the 250 m resolution hillshade available for the study area (Figures 6.13 and 6.14). For the second set of graphs, we specifically design a new color palette `ygb.c`, using the `colorRampPalette()` command, which allows us to assign colors to a palette. We then assign the number of color shades to be generated and the command interpolates between the assigned core colors.

```
> hill_250m_utm <- raster("raster/topo/hill_250m_utm.tif")
> par(mfcol=c(2,2))
> plot(hill_250m_utm,col=grey(0:100/100), legend=FALSE, axes=F,
ext=extent(SAVI), main ="NDVI")
> plot(NDVI,col=rev(terrain.colors(20,alpha=0.6)),add=T)


> plot(hill_250m_utm,col=grey(0:100/100), legend=FALSE, axes=F,
ext=extent(SAVI), main ="SR")
> plot(SR,col=rev(terrain.colors(20,alpha=0.6)),add=T)


> plot(hill_250m_utm,col=grey(0:100/100), legend=FALSE, axes=F,
ext=extent(SAVI), main ="NDWI")
> plot(NDWI,col=rev(topo.colors(20,alpha=0.6)),add=T)
```

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

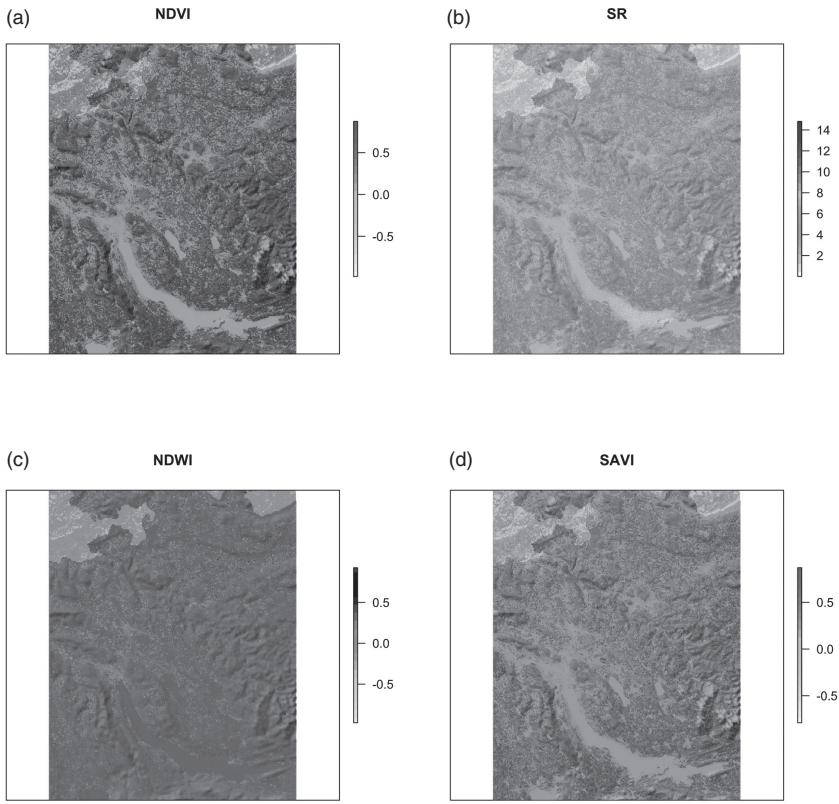


Figure 6.13 Visualization of four band-derived indices: (a) NDVI, (b) SR, (c) NDWI, and (d) SAVI. (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

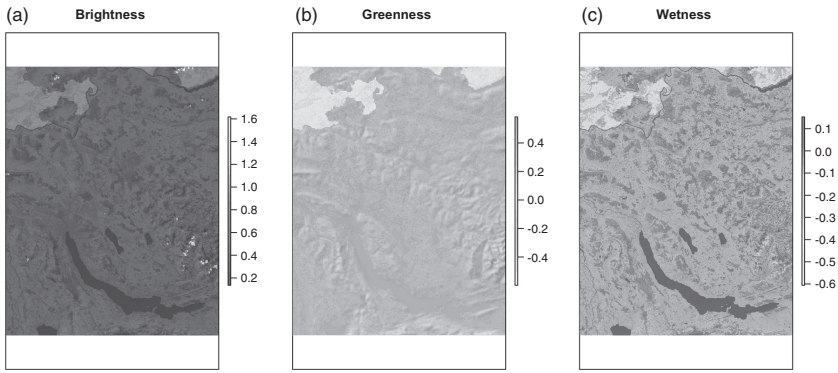


Figure 6.14 Visualization of the three tasseled cap indices: (a) brightness, (b) greenness, and (c) wetness. (*A black and white version of this figure will appear in some formats. For the color version, please refer to the plate section.*)

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

```
> plot(hill_250m_utm,col=grey(0:100/100), legend=FALSE, axes=F,
ext=extent(SAVI), main ="SAVI")
> plot(SAVI,col=rev(terrain.colors(20,alpha=0.6)),add=T)
> par(mfcol=c(1,1))


> ygb.c <-colorRampPalette(c("yellow","#7FFF7F","forestgreen",
"deepskyblue4","#00007F"))

> par(mfcol=c(1,3))
> plot(hill_250m_utm,col=grey(0:100/100), legend=FALSE, axes=F,
ext=extent(SAVI), main ="Brightness")
> plot(L7_010824_Brightness,col=rev(paste(ygb.c(20),"B3",
sep="")),add=T)

> plot(hill_250m_utm,col=grey(0:100/100), legend=FALSE, axes=F,
ext=extent(SAVI), main ="Greenness")
> plot(L7_010824_Greenness,
col=rev(terrain.colors(20,alpha=0.6)),add=T)


> plot(hill_250m_utm,col=grey(0:100/100), legend=FALSE, axes=F,
ext=extent(SAVI), main ="Wetness")
> plot(L7_010824_Wetness,
col=rev(topo.colors(20,alpha=0.6)),add=T)
> par(mfcol=c(1,1))
```

6.4 Properties and Selection of Variables

In this section, we discuss issues relating to the choice of variables and what this means for modeling species' habitat suitability. Since the predictive variables for spatial habitat suitability modeling need to be made available as maps in the form of raster or vector layers (or data frames), it is important to first resolve a number of issues when selecting these variables for modeling.

6.4.1 Accuracy vs. Mechanistic Explanation of Predictors

On the one hand, we need our predictor maps to be accurate, and on the other we want them to closely relate to the true drivers of species ranges, i.e. there is a mechanistic relationship between the predictor and the distribution of our target species. Many predictors are either modeled or derived from topography maps (elevation, see Section 6.2.3). This means that deriving more mechanism-oriented variables (e.g. relative humidity, vapor pressure deficit, frequency of frost days, or plant available soil moisture) requires advanced spatial modeling that naturally includes error propagation (Barry and Elith, 2006). Such maps become more "relevant" and more "mechanistically meaningful" for species modeling, while

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

accumulating a higher rate of propagated error compared to using simple topography maps such as elevation, slope, or aspect. However, using simple topographic variables only produces accurate models if we project our maps to the same, comparably small study, area used to calibrate the models, and if we assume no change in environmental conditions.

One example which clearly illustrates this is the question whether one should use elevation or temperature to model species' ranges. Most species show distinct elevation patterns, despite the fact that elevation (units of meters or feet) has no direct, physiological, or otherwise mechanistic effect on species' ranges (Körner, 2007). However, changes in physiologically and ecologically important variables strongly correlate with changes in elevation, and these variables do have a more direct effect on species and can therefore be more meaningfully used for range delimitation. One clear example is temperature, but other variables are also strongly, and more or less linearly (positively or negatively), correlated with elevation, including precipitation, global radiation, wind speed, and potential evapotranspiration. Other variables often show a hump-shaped relationship with elevation, such as cloudiness. All these variables can affect species' ranges in one way or another, and more or less directly. So using annual mean temperature usually has almost the same predictive power as elevation, since the two are very strongly correlated, but using temperature makes a model more applicable to other areas or time periods. Yet temperature is derived from elevation (and other spatial covariates such as longitude or latitude, distance to lakes, etc.) often using geostatistical methods (Daly et al., 1994; Hutchinson, 1995; Thornton et al., 1997), and therefore includes more propagated error compared to elevation or other simple derivatives. New approaches to mapping temperature from field measurements, topography, and imagery use predictive models instead of geostatistical interpolations (Pradervand et al., 2014).

If the study area becomes (too) large, then indirect or distal variables such as elevation or aspect (see Section 4.1 or Guisan and Zimmermann, 2000) become ineffective as predictors, since they have no mechanistic effect on species distribution, but are only locally correlated with such variables. At such larger scales, more direct variables become more effective predictors (Guisan and Hofer, 2003), despite the higher level of uncertainty associated with them. This is due to their higher relevance and the fact that they are better correlated with the mechanisms (e.g. climatic control) that drive the species range patterns than indirect variables (such as elevation). For instance, the climatic treeline elevation changes very little within a small region of say 20 × 20 km, if there are no strong

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

climate gradients in this study area. Using elevation as a predictor thus might even better explain the treeline position than temperature. Across the globe, treeline elevation varies between 0 and 4800 m a.s.l., while the summer mean temperature only varies within the very small range of error in soil temperature of 0.8°C for the globe (6.7±0.8°C; see Körner and Paulsen, 2004). Therefore, soil temperature is a good predictor of treeline position.

This shows that, as far as possible, we need to avoid using indirect variables, unless our study area is small, and our goal is simply to generate highly accurate predictions of current patterns under current environmental conditions. If we used indirect predictors, we would not be able to interpret what drives the spatial distribution of our species. In turn, if we are interested in interpreting our habitat suitability model with regards to the likely drivers of spatial patterns, then we should select more direct and resource (also termed "proximal") variables that have a known mechanistic or direct impact on processes that shape ranges (Austin, 2002, 2007). Such variables are also more likely to remain important under changing environmental conditions and/or when projecting models to new areas (see Part V).

Such considerations are not only relevant for climate variables, but hold for any type of predictor variable, including remotely sensed products or variables related to soils, habitats, or geology. When using RS imagery, one can simply use raw individual bands, irrespective of whether they represent information with close links to mechanistic processes or not (i.e. "distal" variables). RS imagery can also be used to retrieve vegetation structure, or water or nutrient status of the vegetation surface (more "proximal" variables).

6.4.2 Correlation, Collinearity, and Variance Inflation

Most statistical techniques will struggle to successfully fit a stable model if the predictor variables are highly cross-correlated, as this results in multicollinearity issues (see Part III). When GIS was first developed as a tool for spatial modeling, researchers simply obtained and used whichever map was available for modeling. Nowadays, we have a vast array of available maps, be climatic, topographic, or remotely sensed. Since many maps are derived from, or modeled with, a DEM, many of the derived maps reveal strong correlation with elevation, but also correlate with each other. It is therefore important to assess pairwise correlation between variables prior to use in any model fitting. Here, we illustrate how correlations among variables can be visualized. We use a predefined ascii file of

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

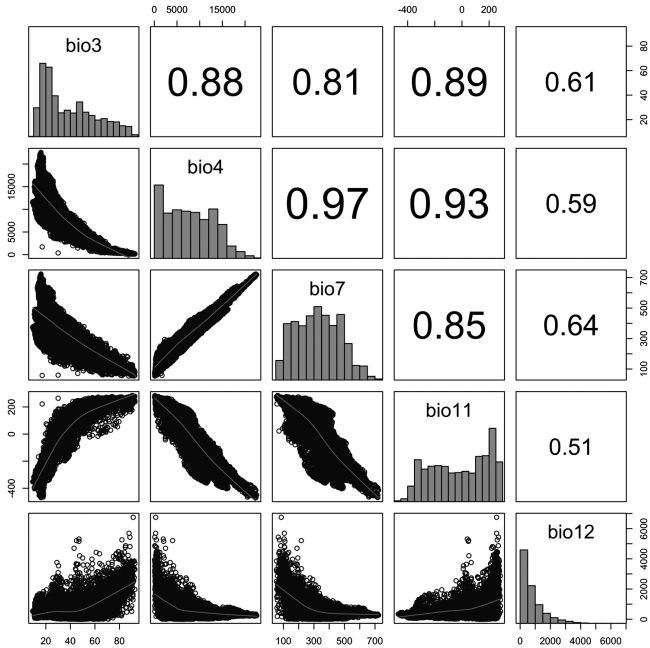


Figure 6.15 Correlation and distribution plot of five bioclim predictor variables. Positive and negative correlations are illustrated in black and red (shown as dark gray), respectively. Weaker correlations are displayed in smaller fonts. The distribution of each variable is illustrated on the diagonal.

bioclim variables (bioclim_table.csv) and the function `ecospat.cor.plot()` from the `ecospat` library, which is based on the `pairs()` command in R. This perfectly illustrates the correlation structures and variable distributions (Figure 6.15, note that it is printed in gray).

```
> data<-read.csv("tabular/bioclim/current/bioclim_table.csv",
header = TRUE, sep = ",")
> ecospat.cor.plot(data[,4:8])
```

This overview obtained using the `ecospat.cor.plot()` function provides us with some interesting information, regarding the five bioclimatic Worldclim[36] variables used. First, bio4 (temperature seasonality) and bio7 (annual temperature range) are strongly correlated (r = 0.97) at global scale. Bio4 is also strongly correlated with bio11 (mean temperature of coldest quarter). Furthermore, we see that bio12 in particular shows considerable skew, while bio7 seems more normally distributed.

[36] www.worldclim.org

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

One way of reducing multicollinearity problems is to remove variables that are too strongly correlated. There is no consensus as to what is an acceptable threshold from a statistical point of view. Numerous papers using distribution models refer to a threshold of r = 0.8 as recommended by Menard (2002), while others refer to a threshold of 0.7 as recommended by Green (1979). However, there is sometimes a hidden correlation structure that is not clearly visible in the pairwise correlation analysis. It can only be detected with a variance inflation factor (VIF) analysis (Harrell, 2001; Guisan et al., 2002, 2006b; Hair et al., 2006). VIF estimates the severity of the effect of multicollinearity, by measuring the extent to which variance in a regression increases due to collinearity compared to when uncorrelated variables are used. VIF tests are recommended, especially when numerous variables are added to a regression, as they detect the variables' linear correlation structure. We can do so, for example, using the `data` object which we have just imported in combination with the `vif()` command from the `usdm` package. Several packages contain a variance inflation test. The `usdm` package has several advantages for our purposes, as it is designed for habitat suitability model testing. It can be used to test for both data frames and raster stack objects, for example. It also offers options for testing which variable combination remains below a given correlation threshold.

```
> library(usdm)
> vif(data[,4:8])
  Variables       VIF
1      bio3  6.813542
2      bio4 63.384660
3      bio7 32.810217
4     bio11 11.786373
5     bio12  2.168148
```

We can see that almost all the variables are above a value of 10.0. Usually values from 5 to 10 are considered as critical for multi-variable correlation. Some authors suggest that VIF values of up to 20 can be accepted, but we do not recommend going above 10. Specifically, we see that bio4 has a very high VIF value. We next test what happens if we remove bio4 from our analyses, the same variable that showed extremely high correlation with several other variables in Figure 6.15.

```
> vif(data[,c(4,6:8)])
  Variables      VIF
1      bio3 5.866959
2      bio7 4.678556
3     bio11 6.881024
4     bio12 1.933955
```

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

We can now see that all values are near or below a VIF value of 6, which we can consider as acceptable for further analyses. We would obtain a similar result if we let the `usdm` package do a stepwise elimination of highly inflating variables using the `vifstep()` command, which applies a default threshold of 10.

```
> vifstep(world.stk[[2:5]])
```

We actually get a very similar result when using the `world.stk` object (only layers 2–5, excluding elevation), which has the same variables stacked as a raster stack. The values are not identical, because the stack and the data frame do not have exactly the same dimension or resolution.

```
> vif(world.stk[[2:5]])
  Variables      VIF
1      bio3 5.815840
2      bio7 4.576193
3     bio11 6.935333
4     bio12 1.931942
```

We can now check which variables remain when only a certain level of correlation is accepted, say r = 0.7.

```
> vifcor(data[,4:8], th=.7)
3 variables from the 5 input variables have collinearity
problem:

bio4 bio11 bio7

After excluding the collinear variables, the linear correlation
coefficients range between:
min correlation ( bio12 ~ bio3 ):  0.6021647
max correlation ( bio12 ~ bio3 ):  0.6021647

---------- VIFs of the remained variables --------
  Variables      VIF
1      bio3 1.568879
2     bio12 1.568879
```

In our case, we can see that only bio3 and bio12 remain at a correlation threshold of 0.7 (which we can also see from Figure 6.15). However, `vif()` does not calculate in the same way as bivariate correlations. VIF is based on the square of the multiple correlation coefficients resulting from regressing a predictor variable against all other predictor variables. It therefore detects multicollinearities that cannot always easily be detected with a simple `pairs()` scatterplot correlation.

More generally speaking, we might ask why we should be concerned about correlations. On the one hand, some statistical methods will fail to

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

correctly fit the influence of the selected variable and thus return biased coefficients. This might strongly influence the importance of variables in the models. It can also be problematic when a model is applied to a changing climate, if the temperature variable has a biased regression coefficient, for example (Dormann et al., 2013). Another, and perhaps even bigger, problem is the fact that the correlation structure among climate (or other) variables as a whole might change significantly in the future. Having too many highly correlated variables included in a regression model (Part III) will tightly constrain the simulated distribution of a species to this correlation structure, and possibly result in false projected extinction rates, should this variable correlation structure change in the future. It is therefore safer and more prudent to avoid selecting highly correlated variables for habitat suitability modeling.

6.4.3 Variable Pre-Selection

Given the wealth of spatial data available today, selecting the right variables for a model is somewhat arbitrary. We have dozens if not hundreds of candidate variables available, and therefore need to decide which ones should be included in a model. This is known as the pre-selection of variables, as most statistical methods will then either down-weight the variables that are unimportant for a certain species (e.g. shrinkage rules), or use statistical variable selection procedures to remove these unimportant variables (e.g. stepwise selection). In order to come up with a pre-selection of variables, we can either use statistical or conceptual reasoning. Statistical reasoning would involve evaluating whether there are any variables that contribute so little to explaining the spatial distribution of a target species that they are insignificant or not much different from a random variable. Common practice to produce a statistically based pre-selection is to select the most important variables, and from these produce a selection that is not too strongly correlated and which passes a VIF test.

Alternatively, one could start with conceptual reasoning and ecological theory (Austin, 2002, 2007). We might, for example, ask: "*what is primarily constraining the niche of a species, and thus its spatial distribution?*" We can use results from experiments, or more conceptual considerations (see Part I). It still makes sense here to check the correlation structure because when designing a conceptually sound variable pre-selection, we may still pre-select some highly correlated variables (e.g. summer mean temperature and minimum temperature of the coldest month), because they both seem equally important for many species.

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011

If we include both (which we do not recommend in most cases), we need to accept the risk of associated multicollinearity problems with this variable choice. Overall, the pre-selection of variables, and the consideration of correlation or accuracy vs. proximal influence is an open, and still poorly explored field. However, from the content discussed above, it seems clear that a priori using a larger set of predictors (such as all 19 bioclimatic variables of Worldclim) without further considering correlation or their relationship to known drivers of the spatial distribution of species is not an appropriate strategy for the purposes of habitat suitability modeling.

One understudied and underrepresented aspect is the transformation of variables. Some methods primarily expect the response (dependent) variable to follow certain distributions, and several model families have been developed to cope with non-normal distributions. However, it is often beneficial for the analysis to transform predictor. The correlation between variables bio7 and bio12 is e.g. quite a bit smaller if the left-skew is treated with log-transformation.

```
> cor(data$bio12,data$bio7)
 [1] -0.6439925

> cor(log(data$bio12+.001),data$bio7)
 [1] -0.4410228
```

We see quite a large change in correlation (r = -0.44 with log-transformation, r = -0.64 without transformation for bio12), yet we do not see a significant increase in predictive power in all cases where we transformed predictor variables. In essence, transformation of variables has to be tested individually. Note that a small value (0.001) had to be added to the bio12 variable in order to avoid errors from log(0) computations.

Downloaded from https://www.cambridge.org/core. on 12 Feb 2018 at 17:52:51, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/9781139028271.011